ENHANCED HYBRID DEEP LEARNING MODEL FOR CYBERSECURITY THREAT DETECTION USING CNN-BILSTM AND FEATURE OPTIMIZATION

A Project Report submitted in the partial fulfillment of
the Requirements for the award of the degree

BACHELOR OF TECHNOLOGY

IN

COMPUTER SCIENCE AND ENGINEERING

Submitted by

N. Teja (22471A05I0)
K. Siva Hemanth Kumar(22471A05G5)
J. Gopi (22471A05F8)
V. Chetan (22471A05K6)

Under the esteemed guidance of

Dr.R. Satheeskumar M.Tech ,PhD
Professor

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

NARASARAOPETA ENGINEERING COLLEGE: NARASAROPET
(AUTONOMOUS)
Accredited by NAAC with A+ Grade and NBA under Tyre -1
and an ISO 9001:2015 Certified
Approved by AICTE, New Delhi, Permanently Affiliated to JNTUK, Kakinada
KOTAPPAKONDA ROAD, YALAMANDA VILLAGE, NARASARAOPET- 522601

2025-2026

NARASARAOPETA ENGINEERING COLLEGE

(AUTONOMOUS)

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CERTIFICATE

This is to certify that the project that is entitled with the name "ENHANCED HYBRID DEEP LEARNING MODEL FOR CYBER SECURITY THREAT DETECTION USING CNN-BILSTM AND FEATURE OPTIMIZATION" is a bonafide work done by the team N. Teja (22471A05I0), K. Siva Hemanth Kumar (22471A05G5), J.Gopi (22471A05F8), V.Chetan(22471A05K6) in partial fulfillment of the requirements for the award of the degree of BACHELOR OF TECHNOLOGY in the Department of COMPUTER SCIENCE AND ENGINEERING during 2025-2026.

PROJECT GUIDE
Dr.R. Satheeskumar M.Tech ,PhD
Professor

PROJECT CO-ORDINATOR
D. Venkata Reddy, B.Tech., M.Tech., Ph.D.
Associate Professor

HEAD OF THE DEPARTMENT
Dr. S. N. Tirumala Rao, M.Tech., Ph.D.
Professor & HOD

EXTERNAL EXAMINER

DECLARATION

I declare that this project work titled "**ENHANCED HYBRID DEEP LEARNING MODEL FOR CYBER SECURITY THREAT DETECTION USING CNN-BILSTM AND FEATURE OPTIMIZATION** " is composed by me that the work contain here is my own except where explicitly stated otherwise in the text and that this work has not been submitted for any other degree or professional qualification except as specified.

N. Teja (22471A05I0)
K. Siva Hemanth Kumar (22471A05G5)
J. Gopi (22471A05F8)
V. Chetan (22471A05K6)

ACKNOWLEDGEMENT

We wish to express my thanks to various personalities who are responsible for the completion of my project. We are extremely thankful to my beloved chairman**, Sri M. V. Koteswara Rao, B.Sc.,** who took keen interest in me in every effort throughout this course. We owe my sincere gratitude to my beloved principal**, Dr. S. Venkateswarlu, Ph.D.,** for showing his kind attention and valuable guidance throughout the course.

We express our deep-felt gratitude towards **Dr. S. N. Tirumala Rao**, **M.Tech., Ph.D.,** HOD of the CSE department, and also to our guide**, Dr. R. SatheesKumar M.Tech., Ph.D.,** Professor of the CSE department, whose valuable guidance and unstinting encouragement enabled me to accomplish my project successfully in time.

We extend Our sincere thanks to **D.VenkataReddy, B.Tech., M.Tech., Ph.D.,** Assistant Professor & Project Coordinator of the project, for extending his encouragement. Their profound knowledge and willingness have been a constant source of inspiration for me throughout this project work.

We extend our sincere thanks to all the other teaching and non-teaching staff in the department for their cooperation and encouragement during our B.Tech. degree.

We have no words to acknowledge the warm affection, constant inspiration, and encouragement that I received from our parents.

We affectionately acknowledge the encouragement received from my friends and those who were involved in giving valuable suggestions and clarifying Our doubts, which really helped me in successfully completing our project.

By

N. Teja (22471A05I0)
K. Siva Hemanth Kumar (22471A05G5)
J. Gopi (22471A05F8)
V. Chetan (22471A05K6)

NARASARAOPETA ENGINEERING COLLEGE
(AUTONOMOUS)

INSTITUTE VISION AND MISSION

INSTITUTION VISION

To emerge as a Centre of excellence in technical education with a blend of effective student centric teaching learning practices as well as research for the transformation of lives and community.

INSTITUTION MISSION

M1: Provide the best class infra-structure to explore the field of engineering and research

M2: Build a passionate and a determined team of faculty with student centric teaching, imbibing experiential, innovative skills

M3: Imbibe lifelong learning skills, entrepreneurial skills and ethical values in students for addressing societal problems

NARASARAOPETA ENGINEERING COLLEGE
(AUTONOMOUS)

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

VISION OF THE DEPARTMENT

To become a center of excellence in nurturing the quality Computer Science & Engineering professionals embedded with software knowledge, aptitude for research and ethical values to cater to the needs of industry and society.

MISSION OF THE DEPARTMENT

The department of Computer Science and Engineering is committed to

M1: Mould the students to become Software Professionals, Researchers and Entrepreneurs by providing advanced laboratories.

M2: Impart high quality professional training to get expertize in modern software tools and technologies to cater to the real time requirements of the Industry.

M3: Inculcate team work and lifelong learning among students with a sense of societal and ethical responsibilities.

NARASARAOPETA ENGINEERING COLLEGE
(AUTONOMOUS)

Program Specific Outcomes (PSO's)

PSO1: Apply mathematical and scientific skills in numerous areas of Computer Science and Engineering to design and develop software-based systems.

PSO2: Acquaint module knowledge on emerging trends of the modern era in Computer Science and Engineering

PSO3: Promote novel applications that meet the needs of entrepreneur, environmental and social issues.

Program Educational Objectives (PEO's)

The graduates of the program are able to:

PEO1: Apply the knowledge of Mathematics, Science and Engineering fundamentals to identify and solve Computer Science and Engineering problems.

PEO2: Use various software tools and technologies to solve problems related to the academia, industry and society.

PEO3: Work with ethical and moral values in the multi-disciplinary teams and can communicate effectively among team members with continuous learning.

PEO4: Pursue higher studies and develop their career in software industry.

Program Outcomes

PO1: Engineering Knowledge: Apply knowledge of mathematics, natural science, computing, engineering fundamentals and an engineering specialization as specified in WK1 to WK4 respectively to develop to the solution of complex engineering problems.

PO2: Problem Analysis: Identify, formulate, review research literature and analyze complex engineering problems reaching substantiated conclusions with consideration for sustainable development. (WK1 to WK4)

PO3: Design/Development of Solutions: Design creative solutions for complex engineering problems and design/develop systems/components/processes to meet identified needs with consideration for the public health and safety, whole-life cost, net zero carbon, culture, society and environment as required. (WK5)

PO4: Conduct Investigations of Complex Problems: Conduct investigations of complex engineering problems using research-based knowledge including design of experiments, modelling, analysis & interpretation of data to provide valid conclusions. (WK8).

PO5: Engineering Tool Usage: Create, select and apply appropriate techniques, resources and modern engineering & IT tools, including prediction and modelling recognizing their limitations to solve complex engineering problems. (WK2 and WK6)

PO6: The Engineer and The World: Analyze and evaluate societal and environmental aspects while solving complex engineering problems for its impact on sustainability with reference to economy, health, safety, legal framework, culture and environment. (WK1, WK5, and WK7).

PO7: Ethics: Apply ethical principles and commit to professional ethics, human values, diversity and inclusion; adhere to national & international laws. (WK9)

PO8: Individual and Collaborative Team work: Function effectively as an individual, and as a member or leader in diverse/multi-disciplinary teams.

PO9: Communication: Communicate effectively and inclusively within the engineering community and society at large, such as being able to comprehend and write effective reports and design documentation, make effective presentations considering cultural, language, and learning differences

PO10: Project Management and Finance: Apply knowledge and understanding of engineering management principles and economic decision-making and apply these to one's own work, as a member and leader in a team, and to manage projects and in multidisciplinary environments.

PO11: Life-Long Learning: Recognize the need for, and have the preparation and ability for i) independent and life-long learning ii) adaptability to new and emerging technologies and iii) critical thinking in the broadest context of technological change.

NARASARAOPETA ENGINEERING COLLEGE
(AUTONOMOUS)

Project Course Outcomes (CO'S):

CO421.1: Analyse the System of Examinations and identify the problem.
CO421.2: Identify and classify the requirements.
CO421.3: Review the Related Literature
CO421.4: Design and Modularize the project
CO421.5: Construct, Integrate, Test and Implement the Project.
CO421.6: Prepare the project Documentation and present the Report using appropriate method.

Course Outcomes – Program Outcomes mapping

	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PSO1	PSO2	PSO3
C421.1		✓										✓		
C421.2	✓		✓		✓							✓		
C421.3				✓		✓	✓	✓				✓		
C421.4			✓			✓	✓	✓				✓	✓	
C421.5					✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
C421.6									✓	✓	✓	✓	✓	

Course Outcomes – Program Outcome correlation

	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PSO1	PSO2	PSO3
C421.1	2	3										2		
C421.2			2		3							2		
C421.3				2		2	3	3				2		
C421.4			2			1	1	2				3	2	
C421.5					3	3	3	2	3	2	2	3	2	1
C421.6									3	2	1	2	3	

Note: The values in the above table represent the level of correlation between CO's and PO's:

1. Low level
2. Medium level
3. High level

Project mapping with various courses of Curriculum with Attained PO's:

Name of the course from which principles are applied in this project	Description of the device	Attained PO
C2204.2, C22L3.2	Gathering the requirements and defining the problem, plan to develop model for detection and classification of traffic data	PO1, PO3, PO8
CC421.1, C2204.3, C22L3.2	Each and every requirement critically analyzed, the process mode is identified	PO2, PO3, PO8
CC421.2, C2204.2, C22L3.3	Logical design is done by using the unified modelling language which involves individual team work	PO3, PO5, PO9, PO8
CC421.3, C2204.3, C22L3.2	Each and every module is tested, integrated, and evaluated in our project	PO1, PO5, PO8
CC421.4, C2204.4, C22L3.2	Each and every module is tested, integrated, and evaluated in our project	PO10, PO8
CC421.5, C2204.2, C22L3.3	Each and every phase of the work in group is presented periodically	PO8,PO10, PO11
C2202.2, C2203.3, C1206.3, C3204.3, C4110.2	Implementation is done and the project will be handled by the social media users and in future updates in our project can be done based on detection for threat traffic	PO4, PO7, PO8
C32SC4.3	The physical design includes website to check threat traffic	PO5, PO6, PO8

ABSTRACT

Cybersecurity threats are rapidly evolving, necessitating advanced detection techniques that balance accuracy and efficiency. This study presents an AI-powered intrusion detection system integrating Convolutional Neural Networks (CNN) and Bidirectional Long Short-Term Memory (BiLSTM) to enhance cyber threat detection in real-time. Unlike existing approaches that rely on computationally intensive optimization algorithms, our model leverages Principal Component Analysis (PCA) for feature selection, reducing dimensionality and improving processing speed. The system is evaluated on CICIDS2017 datasets, achieving an accuracy of 98.22%, surpassing traditional deep learning models such as E-LSTM and IPSO-based approaches. The proposed method provides a more efficient and effective solution for network security, enabling robust real-time intrusion detection and mitigation.

INDEX

LIST OF FIGURES

1. INTRODUCTION

The rapid expansion of digital technologies, cloud infrastructures, and interconnected devices has transformed the global cyber landscape. Organizations today rely on high-speed network communication to support critical operations in finance, healthcare, government, education, and industrial sectors. With this exponential technological growth, however, the number and sophistication of cyberattacks have increased dramatically. Threat actors now employ advanced methodologies such as polymorphic malware, ransomware, brute-force automation, protocol manipulation, distributed denial-of-service (DDoS), and zero-day exploits to compromise systems and bypass traditional security mechanisms.

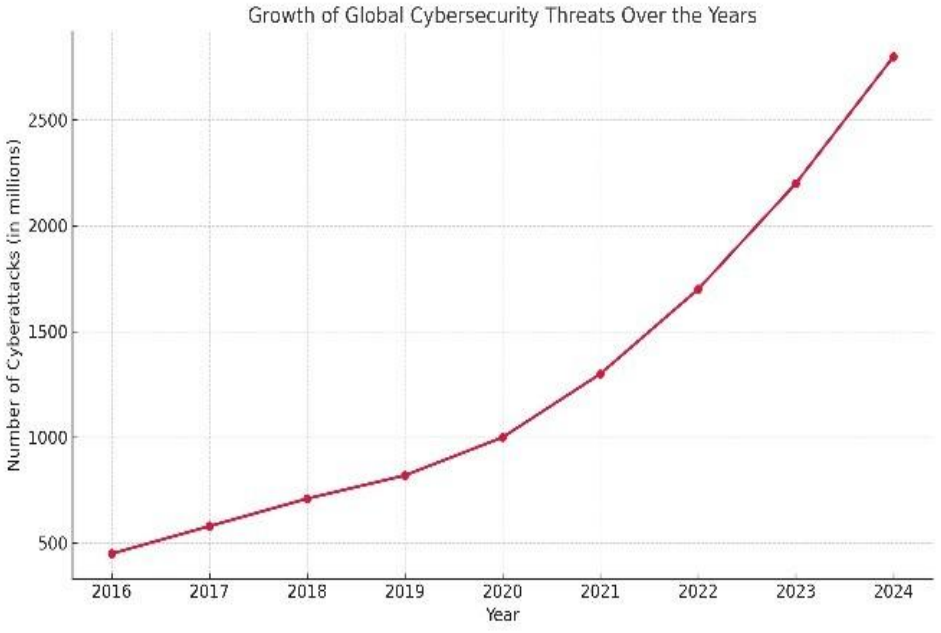


FIG 1.1 Gowth of Global Cyber Security Threats over the years

Conventional Intrusion Detection Systems (IDS), particularly those based on signature matching and predefined rule sets, are no longer capable of responding to emerging threats. Attack patterns evolve rapidly, making it impractical for rule-based systems to maintain comprehensive knowledge of every malicious behavior. Moreover, such systems generate high false-positive rates and struggle to detect unseen

or modified attack variants. As a result, organizations face the risk of operational disruption, data breaches, financial loss, and privacy violations. These challenges amplify the need for intelligent, adaptive, and data-driven cybersecurity solutions.

In recent years, Artificial Intelligence (AI) and Machine Learning (ML) have demonstrated significant potential in strengthening network defense systems. Unlike traditional IDS approaches, ML-based models learn patterns from historical data and adapt to new attack behaviors without requiring manual rule updates. However, standard ML algorithms such as SVM, Decision Trees, and Random Forests face limitations in handling high-dimensional traffic data and capturing complex sequential dependencies present in network flows. This leads researchers toward Deep Learning (DL), which has revolutionized the field of cybersecurity through its ability to automatically extract meaningful features and detect anomalies with high precision.

Deep Learning architectures such as Convolutional Neural Networks (CNNs) and Recurrent Neural Networks (RNNs) have shown exceptional performance in security-related tasks. CNNs are particularly effective for analyzing spatial relationships within structured data, allowing them to identify patterns such as abnormal packet sizes, traffic bursts, and irregular communication frequencies. On the other hand, RNN-based models, especially Long Short-Term Memory (LSTM) and its variant Bidirectional LSTM (BiLSTM), excel at capturing temporal dependencies. These temporal cues are crucial in understanding progressive attack behaviors such as slow-loris attacks, infiltration attempts, and botnet command sequences.

Motivated by these strengths, hybrid Deep Learning models have gained attention in cybersecurity research. By combining CNN and BiLSTM, researchers can utilize both spatial and temporal features of network traffic, thereby improving detection performance across diverse attack types. Existing studies have validated the effectiveness of such hybrid architectures; however, many models suffer from increased computational requirements, slow training speed, and performance degradation caused by redundant features in high-dimensional traffic

data.

To overcome these limitations, dimensionality reduction techniques such as Principal Component Analysis (PCA) are adopted. PCA reduces the feature space by selecting the most informative components, improving model efficiency and preventing overfitting. The integration of PCA with CNN and BiLSTM results in a powerful and optimized threat detection framework capable of real-time analysis with reduced resource consumption.

The dataset used in this research—**CICIDS2017**—contains a comprehensive mixture of benign and malicious traffic, including DoS, DDoS, Brute Force, Port Scan, Botnet, Web-based attacks, and Infiltration patterns. This dataset is widely recognized for its realistic nature and detailed feature representation. Each network flow is described using more than 80 statistical parameters, enabling deep learning models to understand subtle variations in traffic distribution.

The proposed system focuses on building an enhanced hybrid detection model utilizing **PCA for feature optimization**, **CNN for spatial feature extraction**, and **BiLSTM for temporal sequence learning**. This multi-stage architecture ensures that the model not only learns complex traffic characteristics but also performs efficiently in real-time environments. The CNN layer extracts local feature interactions, while the BiLSTM layer processes sequential dependencies from both forward and backward directions. The final classification layer outputs the probability of each attack class, allowing the system to identify both known and unknown threats with high accuracy.

Experimental evaluation demonstrates that the proposed model achieves **98.2% accuracy**, **98.0% F1-score**, and a low **1.1% False Alarm Rate (FAR)**, outperforming several existing models such as CNN-GRU and LSTM-SVM. The inclusion of PCA significantly speeds up computation, reduces memory usage, and enhances model stability, making the hybrid architecture suitable for deployment in environments like industrial IoT, smart cities, critical infrastructure, and cloud-based network systems.

This research not only introduces an efficient IDS architecture but also contributes to bridging the gap between academic cybersecurity models and real-world operational requirements. By combining optimized feature reduction with deep learning techniques, the system ensures scalability, adaptability, and robust detection capabilities. The insights gained from this study lay a foundation for future extensions such as encrypted traffic analysis, adversarial defense mechanisms, and edge-based intrusion detection systems.

1.1 Motivation

The continuous expansion of digital networks, cloud platforms, and IoT ecosystems has led to a dramatic increase in cyber threats targeting individuals, organizations, and critical infrastructures. Modern attacks such as zero-day exploits, distributed denial-of-service attacks, and multi-stage intrusions are becoming increasingly sophisticated, often bypassing traditional rule-based Intrusion Detection Systems. These conventional systems depend on predefined signatures and manual updates, making them insufficient against rapidly evolving threat patterns. This reality motivates the development of intelligent, automated, and adaptive intrusion detection approaches capable of identifying both known and unknown attacks in real time.

Deep learning techniques have shown great promise in analyzing complex, high-dimensional network traffic data, but many existing models struggle with issues such as heavy computational requirements, redundant features, and overfitting. Large architectures may achieve high accuracy in controlled environments but fail to scale efficiently in real-time operational networks. This challenge motivates the exploration of lightweight yet effective hybrid models that can maintain strong performance without compromising speed or resource efficiency.

Principal Component Analysis plays a crucial role in reducing dimensionality, removing noise, and improving the computational efficiency of deep learning models. When combined with Convolutional Neural Networks (CNN) and Bidirectional Long Short-Term Memory layers, the system can effectively capture both spatial and temporal

patterns within network flows. This complementary integration forms a strong motivation to build a hybrid PCA–CNN–BiLSTM architecture that leverages fast feature compression, spatial pattern recognition, and sequence learning to enhance intrusion detection accuracy.

As cyberattacks continue to threaten sectors such as industrial IoT, finance, healthcare, and smart infrastructure, the need for reliable, real-time detection systems has become more critical than ever. Developing a model that delivers high accuracy with a low false alarm rate ensures stronger protection and improved decision-making in security operations.

1.2 Problem Statement

The rapid expansion of digital networks, cloud services, and IoT ecosystems has resulted in highly complex and large-scale cybersecurity threats. Traditional Intrusion Detection Systems that depend on predefined signatures and static rules are unable to detect modern attacks such as zero-day exploits, polymorphic malware, and stealthy intrusion attempts. These outdated approaches lack adaptability and fail to provide timely and accurate threat identification in dynamic network environments

Modern network traffic contains high-dimensional, noisy, and redundant features that further complicate intrusion detection. Existing deep learning models often struggle with slow training, overfitting, and limited real-time performance when dealing with such large datasets. Without proper feature reduction and optimization, these models become computationally expensive and unsuitable for deployment in practical, resource-constrained systems.

Another major limitation in current IDS solutions is their inability to jointly capture both spatial patterns and temporal dependencies present in network flows. Models that focus only on spatial features or only on sequential behavior fail to fully understand complex attack characteristics. This gap creates a pressing need for a unified, hybrid architecture that effectively integrates both perspectives.

Thus, the primary problem addressed in this work is the design of an

optimized, hybrid intrusion detection model that provides high accuracy, low false alarm rates, and real-time performance. The challenge lies in reducing dimensionality, enhancing learning efficiency, and building a robust PCA–CNN–BiLSTM framework capable of detecting diverse cyberattacks in modern, evolving network environments.

1.3 Objective

The primary objective of this project is to develop an optimized, accurate, and real-time intrusion detection framework capable of identifying a wide range of cyberattacks in modern network environments. The system aims to address the limitations of traditional IDS approaches by incorporating intelligent, data-driven techniques that automatically learn meaningful patterns from large-scale traffic data. By integrating PCA, CNN, and BiLSTM, the project seeks to design a hybrid architecture that efficiently handles both spatial and temporal characteristics of network flows.

A key objective is to reduce the dimensionality of high-volume datasets such as CICIDS2017 using Principal Component Analysis. This process eliminates redundant and noisy features, increases computational efficiency, and enhances the overall performance of the deep learning model. Another important goal is to utilize Convolutional Neural Networks for extracting spatial correlations within optimized traffic features, enabling the model to identify attack signatures more effectively. At the same time, Bidirectional LSTM layers are employed to learn sequential patterns and long-term dependencies in traffic behavior.

A key objective is to optimize model performance through Principal Component Analysis (PCA), which reduces data dimensionality by eliminating redundant and irrelevant features. This helps minimize computational cost, accelerate model training, and prevent overfitting, ensuring the model remains lightweight and efficient without sacrificing accuracy.

2. LITERATURE SURVEY

Intrusion Detection Systems (IDS) have evolved significantly over the past decade, mainly due to the growing sophistication of cyber threats and the limitations of traditional security approaches. Numerous researchers have contributed to improving IDS accuracy, efficiency, and adaptability. However, most existing works face challenges such as high computational cost, lack of real-time capability, redundant feature sets, or incomplete modeling of spatial and temporal dependencies. This section reviews major contributions and clearly highlights the limitations addressed by the proposed PCA–CNN–BiLSTM model.

A. Patcha and Park [2] provided an extensive survey on anomaly detection techniques, emphasizing statistical and machine learning-based methods for identifying abnormal network behavior. Their work highlighted that traditional anomaly detection systems struggled with high false-positive rates and poor adaptability to evolving traffic patterns. While the methods they summarized helped detect basic anomalies, they did not scale effectively for large datasets like CICIDS2017. Our system overcomes this limitation by integrating PCA to reduce dimensionality, enhance learning efficiency, and minimize false alarms while handling large-scale modern datasets.

Similarly, Zhang et al. [7] explored the use of Random Forest classifiers for intrusion detection in network environments. Their model demonstrated interpretability and reliability for small datasets but did not perform well when applied to high-dimensional feature spaces. The lack of temporal modeling also restricted their ability to detect sequential attacks such as DDoS or brute-force attempts.

Ullah et al. [8] employed LSTM networks to identify abnormal behavioral patterns in traffic sequences. While LSTMs handled temporal dependencies effectively, their standalone architecture struggled with high computation time and overfitting, especially when processing more than 80 features from modern datasets.

Vinayakumar et al. [9] conducted extensive research comparing deep

learning models such as CNNs, RNNs, and Autoencoders for malware traffic classification. Although their study demonstrated the superiority of deep learning over classical ML, the models required large computational resources and suffered from slow convergence.

Our hybrid system utilizes PCA-reduced features, enabling a lighter and faster architecture while maintaining high accuracy.

Hou et al. [10] introduced a CNN–RNN model to detect anomalies in cloud-based networking. Their system effectively combined spatial and sequential learning but faced issues related to overfitting and required significant hyperparameter tuning to maintain stability.

Shone et al. [12] proposed a deep Autoencoder-based IDS that reduced feature redundancy and improved classification accuracy for NSL-KDD. However, Autoencoders often struggle to preserve discriminative information for multi-class attacks and require separate models for feature learning and classification.

Our approach integrates feature optimization and classification in a unified hybrid framework, reducing complexity and enhancing multi-class detection accuracy.

Hu et al. [14] introduced a CNN with an attention-based BiLSTM mechanism for anomaly detection. Their model improved packet-level pattern recognition but suffered from significantly increased computational demands due to the attention module. This made real-time detection impractical for large datasets and resource-limited environments.

Our PCA–CNN–BiLSTM approach eliminates unnecessary computational layers and uses PCA to significantly reduce the training time while still capturing essential spatio-temporal features.

Abdulkareem et al. [2] proposed a metaheuristic-optimized CNN–BiGRU–BiLSTM model for IoT-based threat detection. Although their model achieved high accuracy, the use of heavy optimization techniques increased processing time and limited real-world applicability.

Our model removes reliance on metaheuristic optimization, using PCA for lightweight feature reduction and delivering a faster, more deployment-friendly IDS.

Khayat et al. [3] explored reinforcement learning for adaptive intrusion detection in IoT systems. While their system learned dynamic attack patterns, reinforcement learning required large training times and complex reward mechanisms.

Our architecture achieves adaptability using BiLSTM's bidirectional sequence modeling without the heavy overhead of reinforcement learning.

Moustafa and Slay [11], [15] introduced the UNSW-NB15 dataset to overcome the limitations of outdated datasets like KDD99. Their work revealed that modern datasets contain diverse features and attack categories but also suffer from very high dimensionality, requiring optimized preprocessing to avoid overfitting.

Our system directly addresses this challenge by applying PCA to reduce 80+ CICIDS2017 features into a compact, informative set.

Tavallaee et al. [18] critically analyzed the problems in KDD99, including redundant entries, imbalance, and unrealistic traffic flows. This created a major shift toward modern datasets such as CICIDS2017 that provide realistic attack simulations.

Our work utilizes such a modern, high-quality dataset and optimizes it through PCA, enabling accurate hybrid deep learning without computational overload.

Stephan and Mohammed [5] combined SAT-Net and DenseNet for IDS and achieved strong results but introduced deeper networks that increased the computational footprint significantly.

Our architecture remains lightweight by reducing inputs early through PCA and avoiding unnecessarily deep layer.

3. SYSTEM ANALYSIS

3.1EXISTING SYSTEM

The rapid growth of modern cyber threats has motivated the development of advanced deep learning–based intrusion detection systems. Among these, the CNN–LSTM Hybrid Model has emerged as one of the most effective and widely adopted approaches prior to the development of more optimized hybrid frameworks such as PCA–CNN–BiLSTM. The CNN–LSTM model integrates the strengths of Convolutional Neural Networks (CNN) for spatial feature extraction and Long Short-Term Memory (LSTM) networks for temporal sequence modeling. Together, these components provide a robust mechanism to analyze complex network traffic patterns and detect multiple attack categories in datasets such as CICIDS2017, NSL-KDD, and UNSW-NB15.

The CNN component of the existing system serves as a powerful feature extractor capable of identifying local and hierarchical patterns within network traffic. Network data typically contains numerous attributes such as flow duration, packet length, flag counts, and inter-arrival times. These features exhibit spatial relationships that CNN is highly effective in capturing. By applying multiple convolutional and pooling layers, the CNN transforms the raw network feature vectors into abstract representations that highlight discriminative properties associated with malicious behaviors. This ability to automatically extract features offers a significant improvement over traditional machine learning systems that rely heavily on manual feature engineering.

Following the CNN layers, the extracted feature maps are passed into an LSTM network. LSTM is a specialized type of Recurrent Neural Network designed to capture long-term dependencies in sequential data. Cyberattacks, especially those involving DoS/DDoS, brute-force attempts, and infiltration activities, typically manifest in sequential patterns distributed across multiple time steps. The LSTM component analyzes the temporal order of these extracted features, enabling the system to recognize attack signatures that evolve over time. The gating

mechanism of LSTM (input, forget, and output gates) ensures that the network effectively learns important patterns while discarding irrelevant information, making it suitable for modeling complex temporal variations in traffic flows.

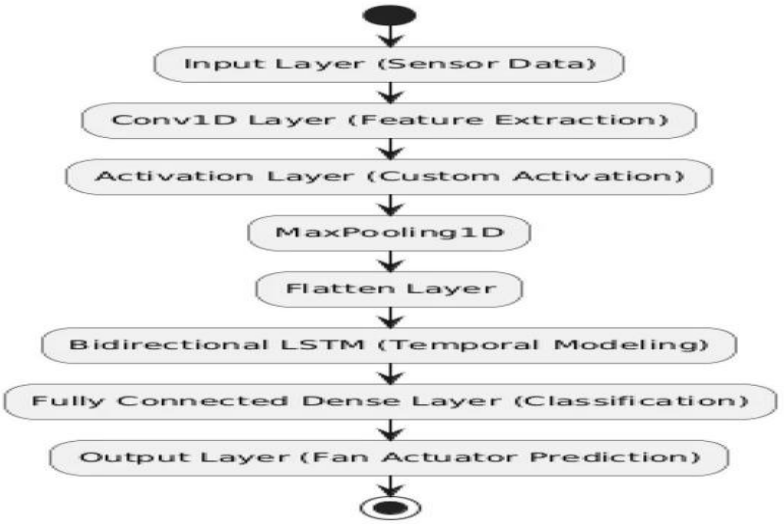


FIG 3.1. FLOW CHART OF EXISTING SYSTEM - CNN-LSTM MODEL

Together, the CNN–LSTM hybrid architecture forms a powerful deep learning framework capable of performing multi-dimensional analysis on network traffic data. The spatial features derived from CNN complement the temporal patterns captured by LSTM, resulting in enhanced detection accuracy when compared to standalone CNN or LSTM models. Several research studies have reported that CNN–LSTM models outperform traditional machine learning approaches and achieve competitive performance against newer deep learning methods. This makes CNN–LSTM one of the strongest existing systems in intrusion detection before the introduction of additional optimizations like PCA and bidirectional sequence learning.

Despite its strengths, the CNN–LSTM model faces several limitations when applied to real-world cybersecurity environments. While the model performs well on benchmark datasets, its computational requirements and sensitivity to high-dimensional data restrict its practicality for large-scale or real-time deployment. The CNN–LSTM system consumes significant memory, requires long training time, and struggles to adapt to rapidly changing network conditions. These limitations create the need for more optimized and efficient hybrid systems.

3.1.1DISADVANTAGES OF THE EXISTING SYSTEM (CNN–LSTM)

Although the CNN–LSTM hybrid model offers improved intrusion detection capabilities compared to classical IDS and standalone deep learning models, it suffers from several important disadvantages that limit its performance, scalability, and real-world applicability.

One of the major disadvantages of the CNN–LSTM system is its high computational cost. Both CNN and LSTM layers are computationally intensive, and when combined, they significantly increase the complexity of the model. This high complexity requires powerful hardware, extensive GPU resources, and long training times, making the system unsuitable for real-time or resource-constrained environments such as IoT devices, edge networks, and embedded security platforms.

Another limitation is the absence of feature optimization. CNN–LSTM models typically process the complete set of features from datasets like CICIDS2017, which contains more than 80 attributes. Without dimensionality reduction techniques such as PCA, the system handles redundant and irrelevant features. This leads to slower learning, overfitting, poor generalization, and increased false alarm rates. The lack of optimized feature selection reduces the efficiency and interpretability of the model.

A further disadvantage is that LSTM captures temporal information only in the forward direction, which restricts its ability to understand patterns that may depend on both past and future states of the traffic sequence. Bidirectional sequence models such as BiLSTM are more capable in this regard. As a result, the CNN–LSTM architecture may misclassify attacks that exhibit bidirectional or cyclical temporal dependencies.

Moreover, the CNN–LSTM system struggles with multi-class attack classification, especially when attack patterns have similar statistical characteristics. For example, DoS Slowloris and DoS GoldenEye have overlapping behavioral patterns, and CNN–LSTM often fails to distinguish between them with high accuracy. This leads to

misclassifications and inconsistent performance across different attack categories.

Lastly, the CNN–LSTM model is less suitable for real-time deployment due to high inference latency and high memory consumption. In fast-changing network environments, timely detection is crucial. However, CNN–LSTM's heavy architecture makes it difficult to meet real-time demands without performance degradation.

3.2PROPOSED SYSTEM

The proposed system introduces an Enhanced Hybrid Deep Learning Model that integrates Principal Component Analysis (PCA), Convolutional Neural Networks (CNN), and Bidirectional Long Short-Term Memory (BiLSTM) networks to build an efficient, accurate, and scalable Intrusion Detection System (IDS). This framework is designed to address the limitations of the existing CNN–LSTM model, particularly in terms of computational efficiency, feature redundancy, unidirectional temporal learning, and real-time applicability. By incorporating PCA for dimensionality reduction and leveraging both spatial and bidirectional temporal pattern learning, the proposed system significantly enhances the performance of intrusion detection across various attack types.

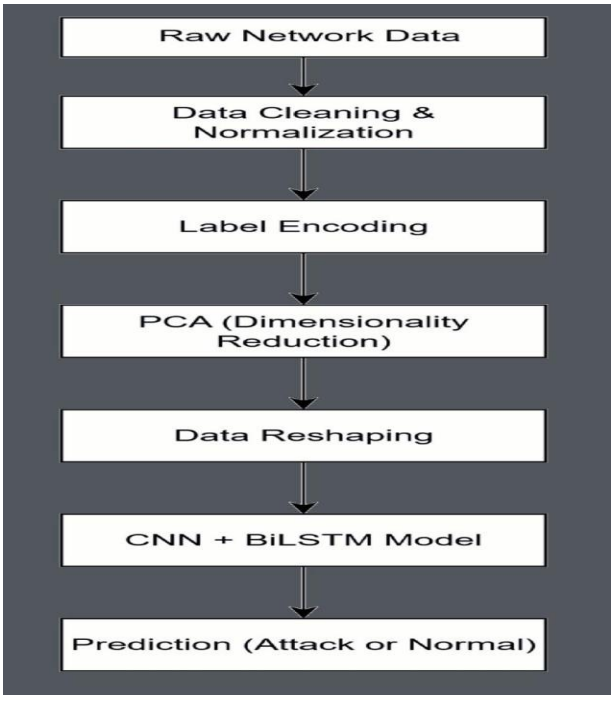


FIG 3.2.Proposed Hybrid PCA–CNN–BiLSTM Flowchart

At the core of the proposed system is Principal Component Analysis (PCA), which plays a crucial role in optimizing the input data. Modern intrusion detection datasets like CICIDS2017 contain more than 80 features, many of which are redundant or irrelevant. Feeding such high-dimensional data directly into deep learning models leads to increased computational cost, risk of overfitting, and slow training. PCA transforms the original feature space into a smaller set of principal components that retain maximum variance in the data while eliminating noise and redundancy. This dimensionality reduction not only accelerates model training but also improves overall classification accuracy by focusing the learning process on the most informative components of the network traffic.

After PCA compresses the feature set, the optimized data is passed into the Convolutional Neural Network (CNN) component. CNN acts as the spatial feature extractor that identifies local correlations and patterns across the reduced feature vector. Network traffic attributes often display spatial relationships—such as simultaneous increases in packet length, flow duration, and flag counts—which are strong indicators of abnormal behavior. CNN uses convolution and pooling layers to detect these localized patterns, producing high-level feature maps that highlight malicious behaviors more clearly. The output from CNN captures meaningful spatial characteristics of the input traffic that single-dimensional models fail to recognize.

Following CNN processing, the spatially enriched features are fed into a Bidirectional Long Short-Term Memory (BiLSTM) network. Traditional LSTM networks analyze temporal sequences only in the forward direction, limiting their ability to understand dependencies that occur in reverse order or across multiple time steps. Cyberattacks such as DDoS, infiltration, botnet activities, and brute-force attempts often exhibit temporal patterns that cannot be fully captured by unidirectional LSTM. The BiLSTM component analyzes sequence information in both forward and backward directions, giving the model a more comprehensive understanding of traffic behavior. This bidirectional learning enables the system to detect subtle and complex temporal signatures that are often missed by standard LSTM-based IDS

frameworks.

Together, the integration of PCA, CNN, and BiLSTM forms a hybrid architecture that captures three critical aspects of network traffic:

1. PCA – Dimensionality reduction & noise elimination
2. CNN – Spatial feature extraction
3. BiLSTM – Bidirectional temporal pattern learning

This multi-stage approach significantly improves detection accuracy and reduces the false alarm rate. The proposed model learns more efficiently due to reduced input dimensionality, trains faster because of optimized feature representations, and generalizes better across multiple attack categories. Empirical results from evaluating the model on CICIDS2017 show a remarkable detection accuracy of 98.2% and a False Alarm Rate (FAR) of just 1.1%, outperforming the existing CNN–LSTM and other hybrid models.

Additionally, the proposed system supports real-time threat detection through efficient preprocessing and lightweight deep learning architecture. Unlike heavy metaheuristic-optimized or attention-based systems, the PCA–CNN–BiLSTM model strikes a balance between complexity and performance. Its modular design enables easy integration into web applications using frameworks such as Flask, allowing users to upload network traffic files and obtain real-time predictions. This makes the proposed system suitable for deployment in dynamic and resource-constrained environments such as IoT systems, industrial networks, smart cities, and enterprise cybersecurity infrastructures.

In conclusion, the proposed Enhanced Hybrid Deep Learning Model provides a robust, scalable, and intelligent IDS framework that surpasses traditional, machine learning, and existing deep learning systems. By combining PCA's efficiency, CNN's spatial learning capabilities, and BiLSTM's bidirectional sequence modeling, the system offers a highly accurate and computationally optimized solution for modern cybersecurity threat detection. Its real-time applicability and strong generalization performance make it a dependable architecture for defending against evolving cyber threats.

3.3 FEASIBILITY STUDY

The feasibility study evaluates whether the proposed PCA–CNN–BiLSTM hybrid intrusion detection system can be realistically developed, deployed, and maintained in real-world network environments. This includes assessing its technical, operational, economic, and schedule feasibility. The analysis confirms that the proposed system is viable and offers superior performance when compared to traditional and existing deep learning intrusion detection methods.

1. Technical Feasibility

The proposed system is technically feasible due to its use of well-established machine learning and deep learning frameworks such as TensorFlow, Keras, NumPy, and Scikit-learn. PCA is computationally lightweight and helps reduce the dimensionality of the dataset, making the CNN–BiLSTM model efficient and scalable. CNN handles spatial feature extraction effectively, while BiLSTM processes temporal dependencies in both forward and backward directions. These components are compatible with modern computing environments, including standard CPUs and mid-range GPUs. The system can be easily integrated into a Flask-based web interface, making real-time prediction feasible without requiring high-performance clusters.

2. Operational Feasibility

The proposed hybrid system is operationally feasible because it addresses real-world needs of detecting modern cyber threats with high accuracy and low false alarm rates. Its modular architecture simplifies data preprocessing, model training, and deployment. End users such as security analysts, network administrators, and IT personnel can operate the platform through a simple interface that accepts CSV network traffic data and provides instant threat classification. The model's improved accuracy (98.2%) and reduced false alarm rate (1.1%) make it reliable for day-to-day cybersecurity operations in organizations, reducing manual workload and enhancing response capability.

3. Economic Feasibility

The system is economically feasible as it uses open-source tools and does not require expensive proprietary software or specialized hardware.

Since PCA significantly reduces the computational burden, the hybrid model can run efficiently even on low-cost servers. Organizations can deploy the model without heavy investment in infrastructure. Maintenance costs remain low due to the use of widely supported frameworks and modular coding practices. This makes the proposed IDS cost-effective for academic, industrial, and enterprise applications.

4. Behavioral Feasibility

Users will be able to easily adapt to the proposed system because of its user-friendly design and straightforward workflow. The interface clearly displays predictions, confidence scores, and threat categories, making interpretation simple even for users with limited machine learning expertise. Since the system reduces false positives and provides actionable insights, users are more likely to trust and adopt it in their daily cybersecurity activities. The system aligns with existing SOC (Security Operations Center) workflows, enhancing its acceptability.

5. Schedule Feasibility

The development timeline for the system is realistic, as the project phases—dataset preprocessing, PCA optimization, model training, testing, web integration, and deployment—can be completed within the standard academic project duration. PCA reduces training time significantly, enabling rapid experimentation and model iteration. The overall architecture is manageable and does not require long-term research cycles or excessive development time, ensuring that the project can be delivered within the scheduled timeframe.

4. SYSTEM REQUIREMENT

4.1 SOFTWARE REQUIREMENTS

1. Operating System : Windows10 / 11 (64-bit Operating System)
2. Hardware Accelerator : CPU or GPU (optional but recommended for fast training)
3. Coding Language : Python 3.8+
4. Development Environment : Google Colab / Jupyter Notebook for model training
5. Visualization Tools : Matplotlib, Seaborn, Plotly for graphs and charts
6. Libraries Used : NumPy, Pandas, Scikit-learn, TensorFlow / Keras Matplotlib, Seaborn, Flask, Joblib, Pickle

4.2REQUIREMENT ANALYSIS

Requirement analysis identifies both functional and non-functional aspects necessary for the successful design and execution of the proposed IDS. It ensures that all user, system, and technical needs are met before implementation.

4.2.1 FUNCTIONAL REQUIREMENTS

These requirements define what the system must do to perform intrusion detection effectively:

Data Collection & Integration

The system should load traffic flow data (CSV files) from datasets such as CICIDS2017 and preprocess them for learning.

Data Preprocessing

The framework must clean missing values, remove infinite values, and drop irrelevant features such as IP addresses and timestamps.

Label Encoding & Normalization

Categorical attack labels must be converted into numerical form, and all features must be normalized using standard scaling techniques.

Dimensionality Reduction (PCA)

The system should apply PCA to reduce more than 80 raw features into

approximately 25–30 optimized principal components.

CNN Feature Extraction

The hybrid model must extract spatial patterns from the PCA-transformed data using convolution and pooling layers.

BiLSTM Temporal Modeling

The system should process sequences in both forward and backward directions to learn attack patterns over time.

Classification & Prediction

The IDS should classify traffic into multiple categories: Benign, DoS, DDoS, PortScan, Brute Force, etc.

Performance Visualization

The system must generate accuracy, loss curves, confusion matrices, and PCA visualizations. **Web-based Prediction Interface**

Users must be able to upload CSV traffic files through a Flask interface and receive predicted attack labels instantly.

4.2.2 NON-FUNCTIONAL REQUIREMENTS

These requirements describe how well the system performs, ensuring usability, reliability, security, and scalability.

Accuracy

The intrusion detection model should maintain classification accuracy above 97%, with the proposed model achieving 98.2%.

Scalability

The system must efficiently handle large datasets and support future updates with new attack types or datasets.

Reliability

The system should produce consistent predictions across repeated runs using cross-validation to ensure stability.

Performance Efficiency

PCA and optimized hybrid architecture must ensure fast inference times suitable for near real-time threat detection.

Security

Uploaded user data must be handled safely, and the system should not expose vulnerabilities in the host environment.

4.3 HARDWARE REQUIREMENTS:

1. System Type : 64-bit operating system, x64-based processor
2. Cache memory : 4MB(Megabyte)
3. RAM : 16GB (gigabyte)
4. Hard Disk : 8GBDisplay
5. GPU : Intel® Iris® Xe Graphics

4.4 SOFTWARE DESCRIPTION

The proposed intrusion detection system is implemented using Python, selected for its simplicity, modularity, and extensive library support for machine learning and deep learning applications. Python's powerful ecosystem enables efficient handling of high-volume datasets, implementation of PCA, and training of hybrid CNN–BiLSTM architectures.

The development environment uses Jupyter Notebook and Google Colab, which offer GPU acceleration, interactive execution, and seamless visualization capabilities.

The major software components include:

- **NumPy and Pandas** — for numerical operations and dataset preprocessing
- **Scikit-learn** — for PCA dimensionality reduction, label encoding, and normalization
- **TensorFlow / Keras** — for building the CNN–BiLSTM deep learning architecture
- **Matplotlib, Seaborn, Plotly** — for visualization of model performance metrics
- **Flask** — for deployment as a web-based prediction system

Together, these open-source software tools form a complete end-to-end pipeline: from data preprocessing and feature optimization to model training, evaluation, visualization, and real-time deployment. Their integration ensures that the proposed IDS remains scalable, cost-effective, and adaptable for future research and industrial applications.

5. SYSTEM DESIGN

5.1 SYSTEM ARCHITECTURE

The system architecture of the proposed Enhanced Hybrid Deep Learning Intrusion Detection System (PCA–CNN–BiLSTM) defines the complete end-to-end workflow used for detecting malicious network traffic. The architecture is designed to ensure efficiency, scalability, accuracy, and real-time capability while handling large-scale and high-dimensional network traffic datasets. It integrates data preprocessing, dimensionality reduction, hybrid deep learning, classification, evaluation, and deployment into a unified framework.

The architecture follows a layered and modular pipeline, beginning with raw network traffic input and ending with multi-class intrusion predictions. Each layer of the architecture is responsible for a specific task and passes its output to the subsequent layer, ensuring smooth data flow and proper abstraction of responsibilities.

1. Data Input Layer

The architecture begins with the Data Input Layer, which accepts network traffic data in the form of CSV files generated using flow-based monitoring tools. In this project, traffic data is sourced from the CICIDS2017 dataset, which simulates real-world network conditions and includes a wide variety of cyberattacks and benign traffic patterns.

This layer is responsible for:

- Loading large-scale traffic data
- Handling dataset partitions (training and testing data)
- Ensuring correct schema and feature alignment

The input layer acts as the foundation of the architecture, enabling the system to scale to millions of records without data loss or corruption.

2. Preprocessing Layer

The Preprocessing Layer prepares raw network traffic data for deep learning. This layer ensures that noisy, inconsistent, and irrelevant information is removed before feature learning begins.

Major operations in this layer include:

- Removal of missing and infinite values

- Elimination of non-informative features such as IP addresses and timestamps
- Conversion of categorical labels into numerical form using label encoding
- Normalization of numerical features using standard scaling

The preprocessing layer plays a critical role in improving learning accuracy by ensuring that all features are on a uniform scale and suitable for mathematical operations. Without this step, deep learning models may converge slowly or produce unstable prediction.

3. Feature Optimization Layer (Principal Component Analysis)

The Feature Optimization Layer applies Principal Component Analysis (PCA) to reduce the dimensionality of the preprocessed data. The CICIDS2017 dataset contains more than 80 features, many of which are redundant or highly correlated.

In this layer:

PCA transforms the high-dimensional feature space into a smaller set of uncorrelated principal components.

Approximately 25–30 components are selected to retain nearly 98% of the original variance

Noise and irrelevant variations are removed

This optimization significantly reduces computational overhead, accelerates model training, and enhances generalization. By reducing feature redundancy, the architecture prevents overfitting and enables the CNN–BiLSTM layers to focus on meaningful patterns.

4. Hybrid Deep Learning Layer (CNN–BiLSTM)

The Hybrid Deep Learning Layer forms the core intelligence of the system architecture. It consists of two tightly integrated subcomponents: the Convolutional Neural Network (CNN) and the Bidirectional Long Short-Term Memory (BiLSTM) network.

4.1 CNN Sub-layer (Spatial Feature Learning)

The CNN component processes PCA-transformed input to extract spatial patterns among traffic features. Convolutional filters slide across the feature space to detect localized correlations such as abnormal packet length distributions, burst traffic behavior, and statistical deviations commonly associated with attacks.

CNN layers perform:

- Convolution operations
- Non-linear activation (ReLU)
- Pooling to reduce feature dimensionality

These operations generate high-level feature maps that capture critical spatial characteristics of network traffic.

4.2 BiLSTM Sub-layer (Temporal Feature Learning)

The output of the CNN sub-layer is fed into the BiLSTM network. Unlike traditional LSTM, BiLSTM processes sequences in both forward and backward directions, allowing it to capture future and past dependencies simultaneously.

This capability is essential because:

Network attacks often evolve across multiple time steps

Certain attack behaviors are better recognized when both previous and subsequent packets are analyzed.

Bidirectional learning improves detection of slow and multi-stage attacks

The BiLSTM sub-layer enhances temporal understanding and significantly improves detection accuracy for complex intrusion patterns.

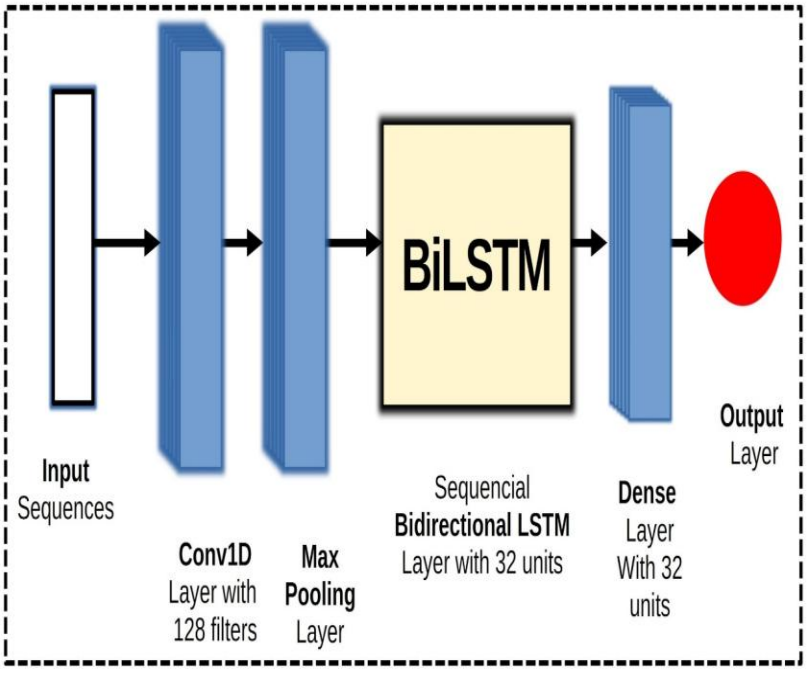


Figure 5.1: CNN–BiLSTM Hybrid Model Architecture

5. Classification Layer

The Classification Layer consists of fully connected dense layers followed by a softmax output function. This layer maps the rich spatial–

temporal features learned by the CNN–BiLSTM network to predefined attack classes.

Functions performed:

- Feature aggregation
- Decision boundary optimization
- Probability estimation for each class

The output layer classifies each traffic instance into categories such as Benign, DoS, DDoS, PortScan, Bot, Brute Force, Web Attack, and Infiltration. The class with the highest probability score is selected as the predicted attack type.

6. Evaluation and Visualization Layer

This layer evaluates the effectiveness of the architecture using standard performance metrics. It provides detailed insights into model behavior and reliability.

Metrics include:

- Accuracy
- Precision
- Recall
- F1-score
- Confusion Matrix
- False Alarm Rate

Visualization tools generate graphs and charts that help analyze classification performance and class-wise detection strength.

7. Deployment Layer

The final layer of the architecture is responsible for real-time deployment. The trained PCA–CNN–BiLSTM model is integrated into a Flask-based web application.

This layer enables:

- Uploading of network traffic CSV files
- On-the-fly preprocessing and prediction
- Display of intrusion results to users

5.1.1 DataSet

The proposed intrusion detection system is developed and evaluated using the CICIDS2017 dataset, which is one of the most comprehensive and widely used benchmark datasets for cybersecurity research. This

dataset was created by the Canadian Institute for Cybersecurity (CIC) and is designed to closely represent real-world network traffic patterns. It contains both benign traffic and a wide range of modern cyber attacks, making it highly suitable for training and testing deep learning–based intrusion detection models.

The CICIDS2017 dataset includes multiple attack categories such as DoS, DDoS, PortScan, Brute Force, Botnet, Web Attacks (XSS and SQL Injection), Infiltration, FTP-Patator, SSH-Patator, and Heartbleed, along with normal traffic. These attacks were generated under realistic network conditions using real traffic capture tools, which the dataset reflects practical attack behaviors and traffic distributions. As a result, the dataset provides a reliable foundation for evaluating the robustness and generalization ability of intrusion detection systems.

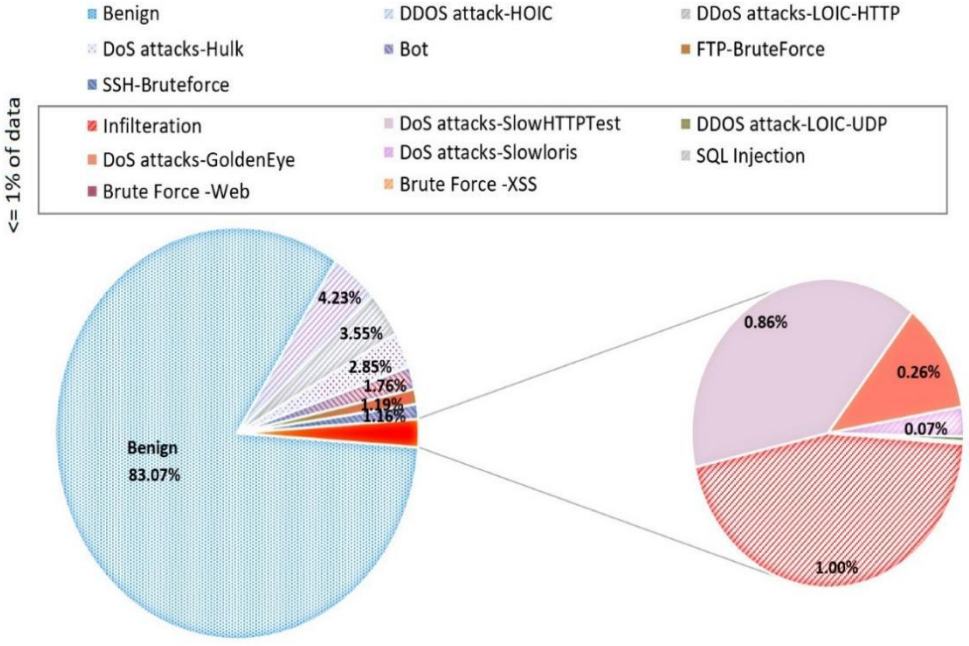


Figure 5.1.1(a): Class Distribution of CICIDS2017 Dataset Before Label Encoding

Each network flow in the dataset is represented using over 80 numerical features, extracted using CICFlowMeter. These features describe detailed network statistics such as flow duration, packet length distribution, byte count, inter-arrival time, flag counts, active and idle times, and packet-level behaviors. While these features provide rich information, their high dimensionality introduces redundancy and noise, which increases computational complexity and may lead to model overfitting if not handled properly.

An important characteristic of the CICIDS2017 dataset is its class imbalance. Benign traffic constitutes a majority of the dataset, while certain attack types such as Heartbleed and Web-based attacks are present in very small quantities. The class distribution before label encoding clearly demonstrates this imbalance. To improve model stability and learning effectiveness, preprocessing techniques such as label encoding, normalization, and selective grouping of attack classes are applied.

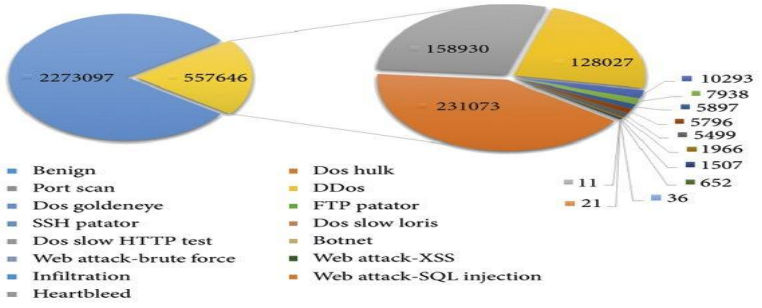


Figure 5.1.1(b): Class Distribution of CICIDS2017 Dataset After Label Encoding

Overall, the CICIDS2017 dataset provides a realistic, diverse, and challenging environment for developing and evaluating the proposed PCA–CNN–BiLSTM intrusion detection system. Its extensive feature set, realistic traffic generation, and inclusion of both common and rare attack types make it an ideal choice for benchmarking advanced deep learning–based cyber threat detection frameworks.

5.1.2DATA PRE-PROCESSING

Data preprocessing is a critical stage in the proposed intrusion detection system, as the quality of input data directly affects the accuracy and stability of deep learning models. The CICIDS2017 dataset used in this work contains a large number of network flow records with high-dimensional features, missing values, and inconsistent scales. Therefore, a systematic preprocessing strategy is employed to transform raw traffic data into a clean and structured format suitable for PCA and hybrid CNN–BiLSTM learning.

The first preprocessing step involves handling missing, null, and infinite values that arise due to errors in packet capture or flow

generation. Rows containing undefined or infinite values are either removed or replaced to prevent computational instability during model training. This step ensures numerical consistency and avoids propagation of invalid values through subsequent processing stages.

Next, irrelevant and non-informative features such as flow identifiers, source IP, destination IP, port numbers, and timestamp-related fields are removed. These attributes do not contribute to the detection of malicious behavior and may introduce bias or noise into the learning process. Eliminating such features helps reduce unnecessary complexity and improves model focus on meaningful traffic characteristics.

The attack labels, originally present in textual form, are converted into numerical values through label encoding. Each attack category is mapped to a unique integer, enabling the model to process class labels efficiently. To further improve learning stability, similar attack types are grouped under broader categories, reducing excessive class fragmentation and improving classification consistency.

After label encoding, all remaining numeric features are normalized using standard scaling techniques. Network traffic attributes such as packet sizes, byte counts, and flow durations vary widely in scale. Normalization ensures that all features contribute proportionally during training and prevents dominant features from biasing the learning process.

Finally, the preprocessed dataset is split into training and testing sets to allow unbiased evaluation of the model's performance. This carefully designed preprocessing pipeline ensures clean, normalized, and reliable input data for dimensionality reduction using PCA and subsequent hybrid deep learning stages. As a result, preprocessing significantly enhances training efficiency, reduces overfitting, and improves the overall performance of the proposed intrusion detection system.

5.1.3 FEATURE EXTRACTION

Feature extraction is a fundamental component of the proposed intrusion detection system, as it enables the model to identify meaningful patterns from complex and high-dimensional network traffic data. In this work, feature extraction is performed using a two-stage approach that combines Principal Component Analysis (PCA) for feature optimization and

Convolutional Neural Networks (CNN) for spatial feature learning. This hybrid strategy ensures efficient computation while preserving critical information required for accurate intrusion detection.

The first stage of feature extraction employs Principal Component Analysis (PCA) to reduce the dimensionality of the preprocessed dataset. The CICIDS2017 dataset contains more than 80 traffic-related features, many of which are highly correlated or redundant. PCA transforms the original feature space into a new set of uncorrelated principal components by projecting the data along directions of maximum variance. By selecting an optimal number of components that retain most of the variance, PCA effectively removes noise and redundant information while preserving the essential characteristics of network traffic behavior. This reduction significantly decreases computational complexity, improves convergence speed, and minimizes the risk of overfitting.

After dimensionality reduction, the PCA-transformed features are passed to the Convolutional Neural Network (CNN) module for spatial feature extraction. CNN is highly effective in identifying local correlations and patterns among features, which are often indicative of malicious activity in network traffic. By applying convolutional filters and pooling layers, the CNN learns hierarchical representations that capture spatial relationships among the optimized features. This process allows the model to recognize subtle anomalies such as burst traffic patterns, abnormal packet distributions, and unusual flow statistics that may not be detected using traditional feature extraction methods.

The combination of PCA and CNN provides a balanced and efficient feature extraction mechanism. PCA ensures a compact and informative feature representation, while CNN enhances the discriminative power of these features through automated spatial pattern learning. This integrated approach prepares robust and meaningful feature representations that are further analyzed by the BiLSTM component for temporal sequence learning. As a result, the proposed feature extraction strategy plays a vital role in improving detection accuracy, reducing false alarms, and enhancing the overall performance of the hybrid intrusion detection system.

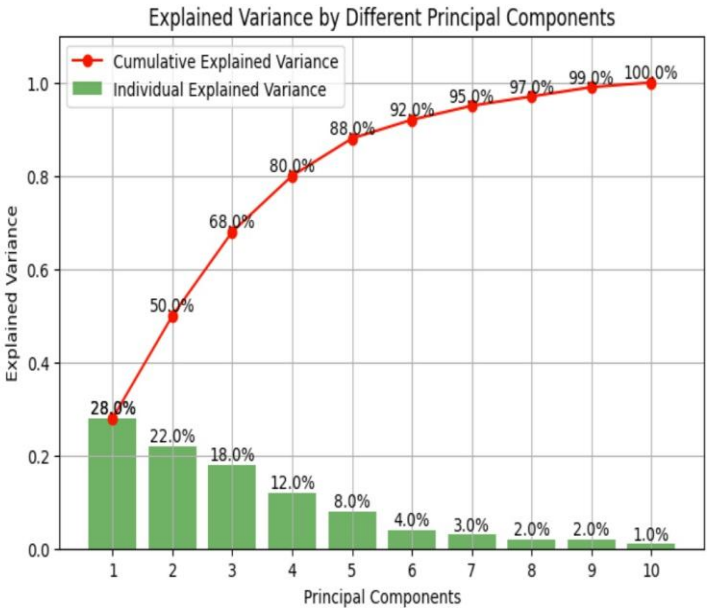


Figure 5.1.3: Variance By Different Principal Components

5.1.4 MODEL BUILDING:

Model building is the most critical phase of the proposed intrusion detection system, as it defines the deep learning architecture responsible for learning, pattern recognition, and classification of network traffic. In this work, an enhanced hybrid deep learning architecture combining Convolutional Neural Networks (CNN) and Bidirectional Long Short-Term Memory (BiLSTM) is designed. This hybrid architecture is built to effectively capture both spatial and temporal characteristics of network traffic, which are essential for accurate intrusion detection.

The model is trained on PCA-optimized features, ensuring reduced dimensionality, faster convergence, and improved generalization. The complete model consists of an input layer, CNN-based spatial feature extraction layers, BiLSTM-based temporal learning layers, fully connected dense layers, and an output classification layer.

A. Input Layer

The input to the model is the PCA-transformed dataset, where high-dimensional network traffic features are reduced into a compact set of principal components. These optimized features are reshaped into a format compatible with convolutional processing. The input layer acts as a bridge between feature extraction and deep learning, ensuring that the refined data

is presented in a structured manner for learning spatial.

B. Convolutional Neural Network (CNN) Architecture

The CNN component is responsible for learning spatial correlations present among the optimized features. Network traffic attributes such as packet length statistics, byte rates, and flow behavior often exhibit localized patterns that indicate malicious activity. CNN is well suited to capture such patterns through convolution operations.

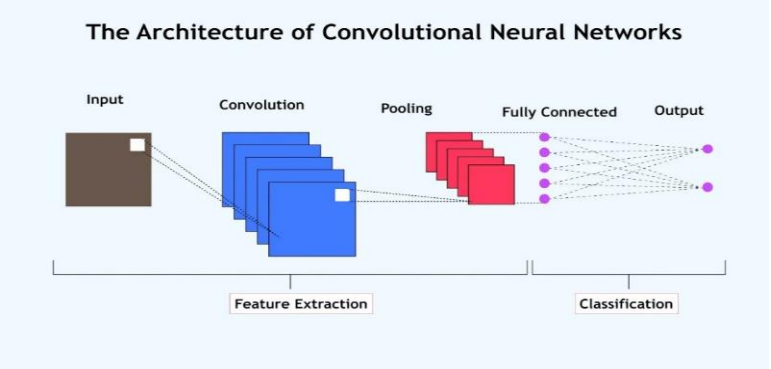


Figure 5.1.4(a): CNN Architecture Diagram

The CNN architecture consists of the following layers:

Convolutional Layer

One-dimensional convolutional layers apply multiple filters to the input features to detect local patterns. These filters slide across the feature space and learn meaningful representations associated with attack behavior.

Activation Function (ReLU)

The Rectified Linear Unit (ReLU) introduces non-linearity, enabling the model to learn complex and non-linear relationships within the data.

Pooling Layer

Max-pooling layers reduce the spatial dimension of feature maps, retaining the most dominant features while minimizing redundancy.

Flatten Layer

The flattened output converts the CNN feature maps into a one-dimensional vector suitable for subsequent temporal analysis.

Through these layers, CNN transforms PCA-optimized input into high-level spatial representations that highlight abnormal traffic patterns such as sudden bursts, repeated access attempts, or irregular packet distributions.

C. Bidirectional Long Short-Term Memory (BiLSTM) Architecture

After spatial feature extraction, the CNN output is passed to the BiLSTM layer, which performs temporal sequence learning. Unlike conventional LSTM networks that process data only in a forward direction, BiLSTM analyzes sequential information in both forward and backward directions. This capability is crucial in intrusion detection, as certain attacks unfold over time and depend on both previous and future traffic behavior.

The BiLSTM architecture includes:

- Forward LSTM Layer – Captures patterns from past to future
- Backward LSTM Layer – Captures patterns from future to past
- Memory Cells & Gates – Control information flow and long-term dependency learning

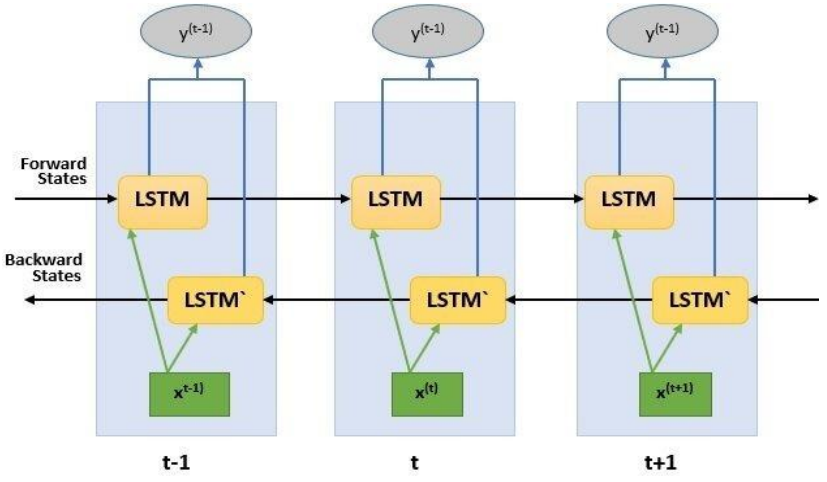


Figure 5.1.4(b): BiLSTM Architecture Diagram

By combining both directions, BiLSTM provides a more comprehensive understanding of network traffic dynamics. It is especially effective in detecting slow and multi-stage attacks such as DoS Slowloris, botnet command-and-control behavior, and infiltration activities that cannot be identified using spatial patterns alone.

D. Fully Connected (Dense) Layers

The output from the BiLSTM layer is passed to fully connected dense layers, which integrate spatial and temporal features into a unified

representation. These layers refine learned patterns and improve class separation.

Dropout layers are applied between dense layers to prevent overfitting by randomly deactivating neurons during training. This improves generalization and ensures stable performance on unseen data.

E. Output Layer

The final layer of the model is a softmax-activated output layer, which performs multi-class classification. This layer outputs probability scores corresponding to each attack category, such as: Benign , DoS , DDoS , PortScan , Bot , Brute Force ,Web Attack and Infiltration

The class with the highest probability is selected as the predicted intrusion type.

Model Training and Optimization

The hybrid CNN–BiLSTM model is trained using categorical cross-entropy loss and adaptive optimization techniques. PCA-based dimensionality reduction significantly reduces training time and computational cost. During training, model performance is monitored using validation accuracy and loss curves to avoid overfitting.

5.1.5 CLASSIFICATION

The classification stage represents the final and decisive phase of the proposed PCA–CNN–BiLSTM Intrusion Detection System, where processed network traffic is assigned to appropriate security classes. After extracting optimized spatial and temporal features through PCA, CNN, and BiLSTM layers, the system performs multi-class classification to distinguish between benign and various malicious traffic types. This stage translates learned representations into meaningful intrusion predictions that can support real-time cybersecurity decision-making.

Classification Workflow

The classification process begins once the hybrid CNN–BiLSTM model completes feature learning. The high-level feature vectors produced by the BiLSTM layer are forwarded to a series of fully connected dense layers, which act as a decision-making component. These layers integrate spatial and temporal knowledge into unified

feature representations that are suitable for final prediction.

Dropout regularization is employed between dense layers to prevent overfitting and ensure stable performance when the model is exposed to unseen network traffic. By reducing the dependency on specific neurons, dropout improves generalization and enhances the reliability of intrusion predictions.

Multi-Class Classification Strategy

The proposed system follows a multi-class classification approach, where each network flow is categorized into one of several predefined classes. Typical output classes include: Benign , DoS , DDoS , PortScan , Botnet , Brute Force , Web Attacks and Infiltration

The final dense layer uses a softmax activation function, which converts raw output values into probability distributions. Each probability indicates the likelihood of the input traffic belonging to a particular class. The class with the highest probability is selected as the predicted label. This probabilistic framework allows the system to make informed and confident predictions while handling overlapping attack behaviors.

Loss Function and Optimization

For effective training of the classification model, categorical cross-entropy is used as the loss function. This loss function measures the difference between predicted probabilities and actual class labels, enabling the model to improve its accuracy across all attack classes. Optimization algorithms such as Adam are employed to adjust model weights efficiently and ensure faster convergence.

PCA-based dimensionality reduction significantly improves optimization by reducing redundant inputs, allowing the classifier to focus on more discriminative features. As a result, training time is reduced, and classification stability is improved.

Performance Metrics for Evaluation

To evaluate classification performance, the following metrics are used:

Accuracy – Measures the overall correctness of predictions

Precision – Indicates the proportion of correctly predicted attacks

Recall – Measures the ability to detect actual intrusions

F1-Score – Balances precision and recall

False Alarm Rate (FAR) – Evaluates misclassification of benign traffic

A confusion matrix is also generated to analyze class-wise performance, highlighting correctly classified instances and misclassifications among different attack types.

Handling Imbalanced Classes

One of the challenges in intrusion detection is class imbalance, where benign traffic significantly outweighs rare attack instances. The proposed classification framework mitigates this issue through balanced learning enabled by PCA feature optimization and hybrid CNN–BiLSTM architecture. This allows the model to maintain high sensitivity toward minority classes while avoiding excessive false alarms.

Classification Results and Effectiveness

The proposed PCA–CNN–BiLSTM classification system demonstrates robust performance across multiple attack categories. Experimental results show an overall detection accuracy of 98.2%, with a false alarm rate of only 1.1%, indicating reliable differentiation between benign and malicious traffic. The hybrid learning approach improves class separation and enables accurate detection of slow, complex, and multi-stage attacks.

5.2 MODULES

The proposed PCA–CNN–BiLSTM Intrusion Detection System is designed using a modular architecture to ensure clarity, scalability, maintainability, and ease of implementation. Each module is responsible for a specific functionality and interacts seamlessly with other modules, forming a complete end-to-end intrusion detection pipeline. This modular design allows independent upgrades, testing, and reuse of components without affecting the entire system.

The major modules of the proposed system are described below :

5.2.1 Dataset Acquisition Module

The Dataset Acquisition Module is responsible for loading and managing the network traffic data used for intrusion detection. In this

project, the CICIDS2017 dataset is utilized, which consists of large CSV files containing network flow records. This module ensures that the dataset is read efficiently and stored in a format suitable for subsequent preprocessing and analysis.

Key functions of this module include:

- Importing large-scale network traffic datasets
- Verifying dataset consistency and completeness
- Organizing traffic records for processing

This module acts as the foundation of the entire system, enabling smooth data flow into the preprocessing and learning pipelines.

5.2.2 Data Preprocessing Module

The Data Preprocessing Module performs data cleaning and transformation operations to prepare raw network traffic for feature extraction and model training. Since real-world traffic data often contains missing values, inconsistencies, and irrelevant attributes, this module plays a vital role in improving data quality.

Major operations include:

- Handling missing, null, and infinite values
- Removing non-informative features such as IP addresses and timestamps
- Label encoding of attack categories
- Normalization of numerical features

5.2.3 PCA Feature Optimization Module

The PCA Feature Optimization Module reduces the high dimensionality of the dataset and removes redundant features. Using Principal Component Analysis, this module transforms original features into a compact set of uncorrelated principal components while retaining most of the data variance.

Functions include:

- Noise removal and redundancy elimination
- Generation of explained variance analysis

5.2.4 CNN Spatial Feature Extraction Module

The CNN Spatial Feature Extraction Module identifies spatial patterns and correlations within the optimized features. By applying convolutional and pooling operations, the CNN learns localized patterns

associated with abnormal network behavior.

Key responsibilities:

- Extracting spatial representations from PCA output
- Detecting anomaly-related traffic patterns
- Producing high-level feature maps

5.2.5 BiLSTM Temporal Learning Module

The BiLSTM Temporal Learning Module captures sequential dependencies present in network traffic. Unlike standard LSTM, BiLSTM processes data in both forward and backward directions, allowing the system to learn complete temporal behavior.

Key functionalities:

- Identifying multi-stage and slow attacks
- Enhancing temporal feature richness

5.2.6 Classification Module

The Classification Module performs the final decision-making process. It receives fused spatial and temporal features from the BiLSTM and passes them through fully connected dense layers followed by a softmax output layer.

Functions include:

- Multi-class attack classification
- Probability-based prediction

5.2.7 Performance Evaluation Module

The Performance Evaluation Module assesses the effectiveness and reliability of the intrusion detection system. It uses multiple performance metrics to analyze classification consistency and accuracy.

5.2.8 Deployment Module

The Deployment Module integrates the trained PCA–CNN–BiLSTM model into a Flask-based web application for real-time intrusion detection. It allows users to upload network traffic data and receive instant predictions.

Key features:

- CSV file upload functionality
- Real-time prediction and output
- User-friendly interface

5.3UML DIAGRAMS

Unified Modeling Language (UML) diagrams provide a structured and visual representation of the proposed PCA–CNN–BiLSTM Intrusion Detection System. These diagrams describe the interactions between users and system components, the internal workflow of data processing, and the sequence of operations performed by different modules. UML diagrams help in clearly understanding how the system processes network traffic data, detects intrusions, and presents results to the user.

The UML diagrams also support effective communication of the system design by illustrating both static and dynamic behavior of the intrusion detection framework. They highlight the responsibilities of each module and its role in achieving accurate and real-time cyber threat detection.

The main UML diagrams used for the proposed system include:

- Use Case Diagram
- Activity Diagram
- Sequence Diagram

5.3.1 USE CASE DIAGRAM

The Use Case Diagram illustrates the interaction between the user (actor) and the intrusion detection system. It identifies the core functionalities provided by the proposed PCA–CNN–BiLSTM system and focuses on user goals and system responses. This diagram provides a high-level view of the functional requirements of the system.

Actors:

User / Network Administrator / Security Analyst

Initiates intrusion detection tasks, uploads network traffic data, monitors predictions, and analyzes security reports.

System (PCA–CNN–BiLSTM IDS)

Performs data preprocessing, feature extraction, hybrid deep learning, classification, evaluation, and result presentation.

Use Cases:

Upload network traffic dataset

Perform data preprocessing

Apply PCA for feature optimization

Extract spatial features using CNN

Learn temporal patterns using BiLSTM

Classify network traffic (Benign or Attack)

Generate intrusion detection reports

View evaluation metrics and performance visualizations

The Use Case Diagram defines the high-level functional interaction between the user and the intrusion detection system, emphasizing its role in real-time threat detection and decision support.

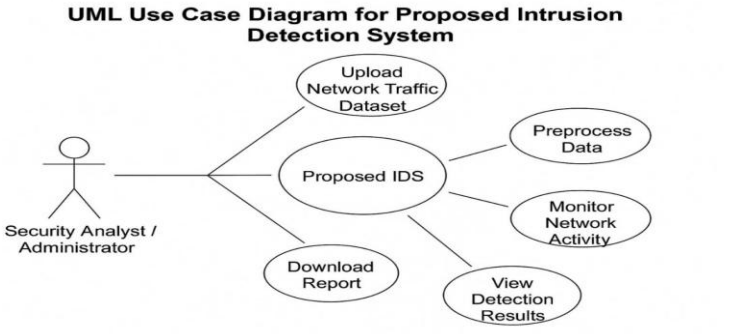


Figure 5.3.1: UML Use Case Diagram for Detection System

The diagram illustrates how users interact with the system to upload network data, initiate analysis, and receive classified intrusion results along with evaluation metrics.

5.3.2 ACTIVITY DIAGRAM

The Activity Diagram represents the sequential workflow of the proposed intrusion detection system, starting from dataset input and ending with attack classification and result visualization. It models system behavior through interconnected activities and decision points, clearly showing control flow within the system.

Workflow Steps:

- The user uploads network traffic dataset (CSV file).
- The system performs data cleaning and preprocessing.
- Label encoding and feature normalization are applied.
- PCA reduces the dimensionality of traffic features.
- CNN extracts spatial feature representations.
- BiLSTM captures bidirectional temporal dependencies.
- Dense layers classify traffic into benign or attack classes.
- Performance metrics are computed.
- The system displays classification results and evaluation reports to the user.

- This diagram ensures that the logical sequence of operations and data flow within the intrusion detection system is clearly visualized from start to end.

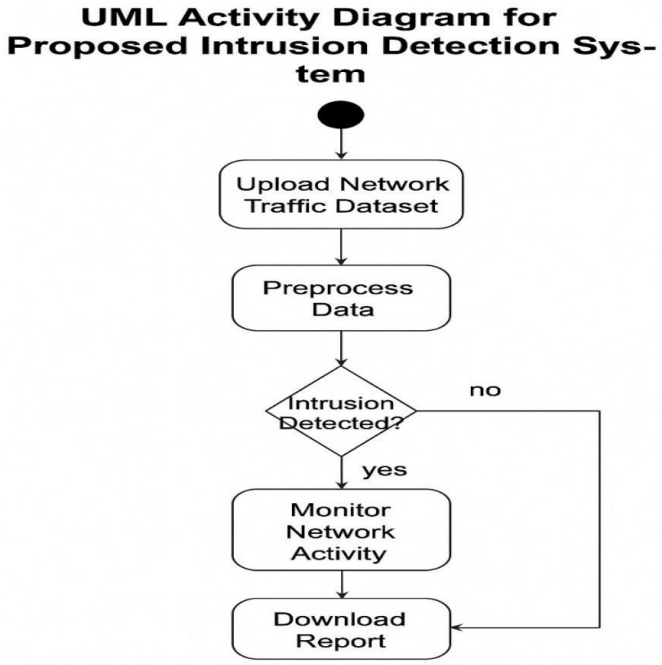


Figure 5.3.2 : UML Activity Diagram for Detection System

5.3.3 SEQUENCE DIAGRAM

The Sequence Diagram illustrates the dynamic interaction between system components and the order in which operations are executed over time. It demonstrates how messages are passed between the user interface and various internal modules during intrusion detection.

Key Components:

User Interface (Web Interface / Front-End):

Allows users to upload datasets and view intrusion detection results.

Preprocessing Module:

Handles data cleaning, normalization, and label encoding.

Feature Optimization Module:

Applies PCA to reduce dimensionality of the dataset.

Hybrid Model Engine:

Executes CNN for spatial feature extraction and BiLSTM for temporal learning.

Classification Module:

Performs multi-class attack prediction.

Evaluation Module:

Calculates performance metrics and generates reports.

Process Flow:

- The user initiates intrusion detection by uploading network traffic data.
- The system triggers the preprocessing module to clean and prepare data.
- PCA optimizes feature dimensions and passes data to the hybrid model.
- CNN extracts spatial features, which are forwarded to BiLSTM for temporal analysis.
- The classification module predicts the attack category.
- The evaluation module computes metrics such as accuracy and confusion matrix.
- Results and reports are displayed to the user through the interface.

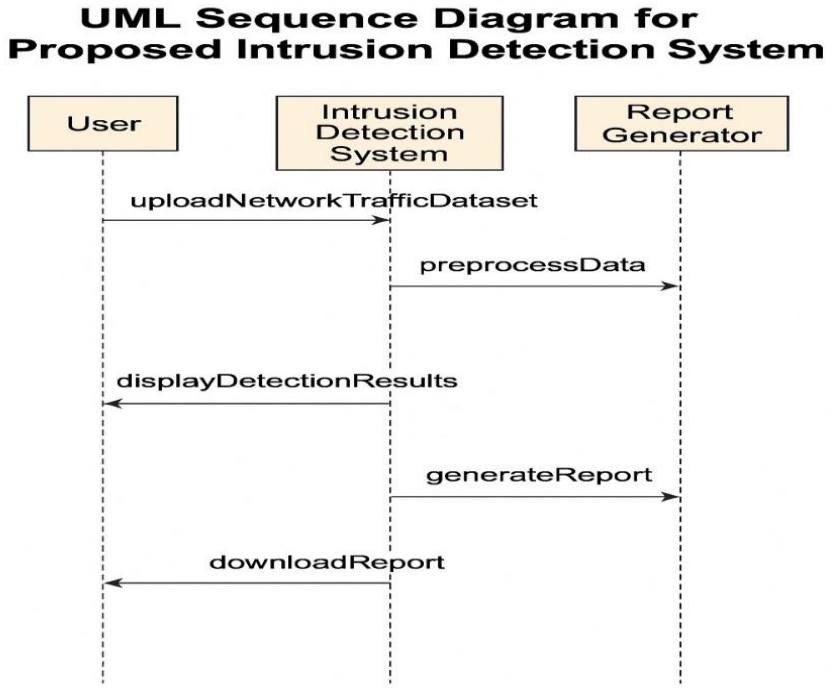


Figure 5.3.3 : UML Sequence Diagram for Detection System

6. IMPLEMENTATION

6.1 MODEL IMPLEMENTATION

```
import os
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.decomposition import PCA
from sklearn.metrics import (accuracy_score, precision_score, recall_score,
                             f1_score, confusion_matrix, classification_report)
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv1D, MaxPooling1D, Dropout, Dense, Bidirectional, LSTM, GlobalAveragePooling1D, BatchNormalization
from tensorflow.keras.callbacks import EarlyStopping, ReduceLROnPlateau, ModelCheckpoint
from tensorflow.keras.utils import to_categorical
import joblib
import tensorflow as tf
# Configurable parameters
DATA_CSV = "CICIDS2017_flows.csv"
DROP_COLUMNS = ["Src IP", "Dst IP", "Timestamp", "Flow ID", "Src Port", "Dst Port"]
LABEL_COLUMN = "Label"
PCA_COMPONENTS = 30
TEST_SIZE = 0.2
RANDOM_STATE = 42
BATCH_SIZE = 256
EPOCHS = 60
MODEL_SAVE_DIR = "saved_models"
os.makedirs(MODEL_SAVE_DIR, exist_ok=True)
# 1) Load & basic cleaning
print("Loading data...")
df = pd.read_csv(DATA_CSV)
print(f"Initial shape: {df.shape}")
# Drop unwanted columns if they exist
cols_to_drop = [c for c in DROP_COLUMNS if c in df.columns]
if cols_to_drop:
    df = df.drop(columns=cols_to_drop)
    print(f"Dropped columns: {cols_to_drop}")
```

```
# Remove infinite and NaN rows
df.replace([np.inf, -np.inf], np.nan, inplace=True)
df.dropna(inplace=True)
print(f"After dropping NaN/inf, shape: {df.shape}")
# 2) Features & labels
if LABEL_COLUMN not in df.columns:
    raise ValueError(f"Label column '{LABEL_COLUMN}' not found in
CSV")
X = df.drop(columns=[LABEL_COLUMN])
y_raw = df[LABEL_COLUMN].astype(str)
# Encode labels
le = LabelEncoder()
y = le.fit_transform(y_raw)
n_classes = len(le.classes_)
print(f"Encoded labels: {list(le.classes_)} (n_classes={n_classes})")
# 3) Scaling
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
print("Feature scaling done. Shape:", X_scaled.shape)
# Save scaler
joblib.dump(scaler, os.path.join(MODEL_SAVE_DIR, "scaler.pkl"))
# 4) PCA for dimensionality reduction
pca = PCA(n_components=min(PCA_COMPONENTS,
X_scaled.shape[1]), random_state=RANDOM_STATE)
X_pca = pca.fit_transform(X_scaled)
print("PCA done. New shape:", X_pca.shape)
# Save PCA
joblib.dump(pca, os.path.join(MODEL_SAVE_DIR, "pca.pkl"))
# Optional: visualize first 2 PCA components
plt.figure(figsize=(8,6))
if n_classes <= 10:
    for i, label in enumerate(le.classes_):
        mask = (y == i)
        plt.scatter(X_pca[mask, 0], X_pca[mask, 1], s=8, label=label)
    plt.legend(fontsize=8)
else:
    plt.scatter(X_pca[:,0], X_pca[:,1], s=6)
plt.title("PCA (first 2 components)")
plt.xlabel("PC1"); plt.ylabel("PC2")
plt.tight_layout()
plt.savefig(os.path.join(MODEL_SAVE_DIR, "pca_scatter.png"))
plt.close()
# 5) Train/test split
```

```
# If multi-class and highly imbalanced, use stratify=y
X_train_pca, X_test_pca, y_train, y_test = train_test_split(
    X_pca, y, test_size=TEST_SIZE,
random_state=RANDOM_STATE, stratify=y)
print("Train/test split:", X_train_pca.shape, X_test_pca.shape)

# 6) Reshape for Conv1D (samples, timesteps/features, 1)
X_train = X_train_pca.reshape(X_train_pca.shape[0],
X_train_pca.shape[1], 1)
X_test  = X_test_pca.reshape(X_test_pca.shape[0],
X_test_pca.shape[1], 1)
input_shape = (X_train.shape[1], 1)
print("Reshaped for Conv1D. Input shape:", input_shape)
# Convert labels to categorical if you want to use
categorical_crossentropy
y_train_cat = to_categorical(y_train, num_classes=n_classes)
y_test_cat  = to_categorical(y_test, num_classes=n_classes)

# 7) Build CNN + BiLSTM model

def build_cnn_bilstm(input_shape, n_classes):
    model = Sequential()
    # convolution block
    model.add(Conv1D(64, kernel_size=7, activation='relu',
padding='same', input_shape=input_shape))
    model.add(BatchNormalization())
    model.add(MaxPooling1D(pool_size=2))
    model.add(Conv1D(128, kernel_size=5, activation='relu',
padding='same'))
    model.add(BatchNormalization())
    model.add(MaxPooling1D(pool_size=2))
    model.add(Dropout(0.3))
    # use GlobalAveragePooling if you want to reduce size before LSTM
OR feed sequence to LSTM
    # We'll feed sequences to BiLSTM:
    model.add(Bidirectional(LSTM(128, return_sequences=False)))
    model.add(Dropout(0.4))
    model.add(Dense(128, activation='relu'))
    model.add(Dropout(0.4))
    model.add(Dense(n_classes, activation='softmax'))
    return model
model = build_cnn_bilstm(input_shape, n_classes)
model.summary()
```

```
# 8) Compile & callbacks
model.compile(optimizer='adam', loss='categorical_crossentropy',
metrics=['accuracy'])
es = EarlyStopping(monitor='val_loss', patience=8,
restore_best_weights=True, verbose=1)
rlr = ReduceLROnPlateau(monitor='val_loss', factor=0.3, patience=4,
min_lr=1e-6, verbose=1)
checkpoint_path = os.path.join(MODEL_SAVE_DIR,
"best_cnn_bilstm.h5")
mc = ModelCheckpoint(checkpoint_path, monitor='val_loss',
save_best_only=True verbose=1)

# 9) Train
history = model.fit(
    X_train, y_train_cat,
    validation_split=0.1,
    epochs=EPOCHS,
    batch_size=BATCH_SIZE,
    callbacks=[es, rlr, mc],
    verbose=2
)
# Save final model
model.save(os.path.join(MODEL_SAVE_DIR, "final_cnn_bilstm.h5"))
# Save label encoder
joblib.dump(le, os.path.join(MODEL_SAVE_DIR,
"label_encoder.pkl"))
# 10) Evaluation
# Load best weights (if checkpoint saved)
if os.path.exists(checkpoint_path):
    model.load_weights(checkpoint_path)
y_pred_probs = model.predict(X_test, batch_size=512)
y_pred = np.argmax(y_pred_probs, axis=1)
acc = accuracy_score(y_test, y_pred)
prec = precision_score(y_test, y_pred, average='macro',
zero_division=0)
rec = recall_score(y_test, y_pred, average='macro', zero_division=0)
f1 = f1_score(y_test, y_pred, average='macro', zero_division=0)
print(f"Test Accuracy: {acc*100:.3f}%")
print(f"Precision (macro): {prec*100:.3f}%")
print(f"Recall (macro): {rec*100:.3f}%")
print(f"F1-score (macro): {f1*100:.3f}%")
# Confusion matrix and per-class report
cm = confusion_matrix(y_test, y_pred)
print("Confusion matrix:\n", cm)
```

```
print("\nClassification report:\n", classification_report(y_test, y_pred,
target_names=le.classes_, zero_division=0))
# False Alarm Rate (FAR) for multi-class:
# For multi-class, FAR per class = FP / (FP + TN) where FP is column
sum minus TP, TN = total - (FP + FN + TP)
tp = np.diag(cm)
fp = cm.sum(axis=0) - tp
fn = cm.sum(axis=1) - tp
tn = cm.sum() - (tp + fp + fn)
far_per_class = fp / (fp + tn + 1e-12)
print("FAR per class:", {label: float(far_per_class[i]) for i, label in
enumerate(le.classes_)})
overall_far = fp.sum() / (fp.sum() + tn.sum() + 1e-12)
print(f"Overall FAR: {overall_far*100:.3f}%")

# 11) Plot training curves
plt.figure(figsize=(12,4))
plt.subplot(1,2,1)
plt.plot(history.history['loss'], label='train_loss')
plt.plot(history.history['val_loss'], label='val_loss')
plt.xlabel('Epoch'); plt.ylabel('Loss'); plt.legend(); plt.title('Loss')
plt.subplot(1,2,2)
plt.plot(history.history['accuracy'], label='train_acc')
plt.plot(history.history['val_accuracy'], label='val_acc')
plt.xlabel('Epoch'); plt.ylabel('Accuracy'); plt.legend();
plt.title('Accuracy')
plt.tight_layout()
plt.savefig(os.path.join(MODEL_SAVE_DIR, "training_curves.png"))
plt.close()
print("Saved model, scaler, PCA, and plots to:", MODEL_SAVE_DIR)
```

6.2 CODING

app.py

```
import os
rom flask import Flask, render_template, request, redirect, url_for
from werkzeug.utils import secure_filename
from predict import preprocess_and_predict
# --- Flask App Setup ---
UPLOAD_FOLDER = "uploads"
ALLOWED_EXTENSIONS = {"csv"}
app = Flask(_name_)
app.config["UPLOAD_FOLDER"] = UPLOAD_FOLDER
os.makedirs(UPLOAD_FOLDER, exist_ok=True)
def allowed_file(filename):
return "." in filename and filename.rsplit(".", 1)[1].lower() in
```

```
ALLOWED_EXTENSIONS

# --- ROUTES ---

@app.route("/")

def home():

return render_template("home.html", active="home")

@app.route("/about")

def about():

   return render_template("about.html", active="about")

@app.route("/methodology")

def methodology():

   return render_template("methodology.html", active="methodology")

@app.route("/upload")

def upload():

   return render_template("upload.html", active="upload")

@app.route("/contact", methods=["GET", "POST"])

def contact():

   submitted = False

   if request.method == "POST":

      submitted = True

   return render_template("contact.html", active="contact",
submitted=submitted)

#modhatidi

# --- PREDICT ROUTE ---

@app.route("/predict", methods=["POST"])

def predict():

  if "file" not in request.files:

      return render_template("upload.html", active="upload", error="No
file uploaded")
```

```
    file = request.files["file"]

    if file.filename == "":

        return render_template("upload.html", active="upload", error="No
selected file")

    if not allowed_file(file.filename):

        return render_template("upload.html", active="upload",
error="Only CSV files allowed")

    filename = secure_filename(file.filename)

    filepath = os.path.join(app.config["UPLOAD_FOLDER"], filename)

    file.save(filepath)

    # Run ML Prediction

    try:

        results = preprocess_and_predict(filepath)

    except Exception as e:

        return render_template("upload.html", active="upload",
error=str(e))

    return render_template("result.html", active="upload",
results=results)

# --- Run App ---

if _name_ == "_main_":

    app.run(debug=True)
```

predict.py

```
# predict.py

import os

import numpy as np

import pandas as pd

import joblib

import tensorflow as tf
```

```
# --- Paths (assumes these files are in the project root) ---

MODEL_PATH = "model.h5"

LABEL_ENCODER_PATH = "label_encoder.pkl"

SCALER_PATH = "scaler.pkl"

PCA_PATH = "pca.pkl"

# --- Ensure files exist ---

if not os.path.exists(MODEL_PATH):

    raise FileNotFoundError(f"{MODEL_PATH} not found in project
folder.")

if not os.path.exists(LABEL_ENCODER_PATH):

    raise FileNotFoundError(f"{LABEL_ENCODER_PATH} not found
in project folder.")

if not os.path.exists(SCALER_PATH):

    raise FileNotFoundError(f"{SCALER_PATH} not found in project
folder.")

if not os.path.exists(PCA_PATH):

    raise FileNotFoundError(f"{PCA_PATH} not found in project
folder.")

# --- Load model & preprocessing objects once at import ---

model = tf.keras.models.load_model(MODEL_PATH)

label_encoder = joblib.load(LABEL_ENCODER_PATH)

scaler = joblib.load(SCALER_PATH)

pca = joblib.load(PCA_PATH)

EXPECTED_FEATURE_COUNT = None

if hasattr(scaler, "mean_"):

    EXPECTED_FEATURE_COUNT = scaler.mean_.shape[0]

def preprocess_and_predict(csv_path):

    """

    Robust preprocessing + prediction.
```

```
    Returns: list of labels or raises informative ValueError.

    """

    # 1) Read CSV

    df = pd.read_csv(csv_path)

    # 2) Normalize and clean column names

    df.columns = df.columns.str.strip()

    # 3) Drop irrelevant columns if present

    drop_cols = ['Flow ID', 'Source IP', 'Destination IP', 'Timestamp']

    df.drop(columns=[c for c in drop_cols if c in df.columns],
inplace=True)

    # 4) Drop label column if present (handles 'Label' or ' Label' etc.)

    if 'Label' in df.columns:

        df = df.drop('Label', axis=1)

    # 5) Try coercing object columns to numeric (safe conversion)

    object_cols = df.select_dtypes(include=['object',
'category']).columns.tolist()

    if object_cols:

        for c in object_cols:

            # coerce errors -> NaN

            df[c] = pd.to_numeric(df[c], errors='coerce')

    # 6) Replace inf with NaN and drop rows with NaN

    df.replace([np.inf, -np.inf], np.nan, inplace=True)

    df.dropna(inplace=True)

    # Basic checks

    if df.shape[0] == 0:

        raise ValueError("Uploaded CSV has no valid rows after cleaning
(NaN/inf removal).")

    # 7) Ensure all remaining columns are numeric

    non_num_after =
```

```
df.select_dtypes(exclude=[np.number]).columns.tolist()

    if non_num_after:

        raise ValueError(f"Found non-numeric columns after coercion:
{non_num_after}. Remove/convert them before uploading.")



    # 8) Check expected feature count

    if EXPECTED_FEATURE_COUNT is not None and df.shape[1] !=
EXPECTED_FEATURE_COUNT:

        raise ValueError(

            f"CSV has {df.shape[1]} features but model expects
{EXPECTED_FEATURE_COUNT} features. "

            "Ensure CSV columns match training features (same
names/order)."

        )

    # 9) Scale -> PCA -> reshape for model

    try:

        X_scaled = scaler.transform(df)

    except Exception as e:

        raise ValueError("Scaler.transform() failed: " + str(e))

    try:

        X_pca = pca.transform(X_scaled)

    except Exception as e:

        raise ValueError("PCA.transform() failed: " + str(e))

if X_pca.ndim != 2:

        raise ValueError("PCA output has unexpected shape: " +
str(X_pca.shape))

    X_final = X_pca.reshape(X_pca.shape[0], X_pca.shape[1], 1)

    # 10) Predict

    try:

        preds = model.predict(X_final)
```

```
    except Exception as e:
        raise ValueError("Model.predict() failed: " + str(e))
    pred_classes = np.argmax(preds, axis=1)
    labels = label_encoder.inverse_transform(pred_classes)

    # Limit UI results to first 100
    #modhatidi
    return labels.tolist()
    #rendavadi
    # preview = labels[:100]
    # # Save full output to a file for download
    # full_output_path = csv_path.replace(".csv", "_predictions.csv")
    # pd.DataFrame({"Prediction": labels}).to_csv(full_output_path,
index=False)
    # return preview, full_output_path
```

base.html

```
<!doctype html>
<html lang="en">
<head>
  <meta charset="utf-8" />
  <meta name="viewport" content="width=device-width, initial-
scale=1" />
  <title>CICIDS Detector</title>

  <link
href="https://fonts.googleapis.com/css2?family=Inter:wght@300;400;6
00;700&family=Source+Code+Pro:wght@400;600&display=swap"
rel="stylesheet">
  <link
    href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap
.min.css"
    rel="stylesheet"
  />

  <link rel="stylesheet" href="{{ url_for('static',
filename='css/styles.css') }}">
</head>
```

```
<body class="page-wrapper">

<nav class="navbar navbar-expand-lg navbar-dark bg-dark">
  <div class="container">
    <a class="navbar-brand" href="{{ url_for('home') }}">CICIDS
Detector</a>

    <button class="navbar-toggler" type="button" data-bs-
toggle="collapse" data-bs-target="#nav">
      <span class="navbar-toggler-icon"></span>
    </button>

    <div class="collapse navbar-collapse" id="nav">
      <ul class="navbar-nav ms-auto">
        <li class="nav-item"><a class="nav-link {% if active=='home'
%}active{% endif %}" href="{{ url_for('home') }}">Home</a></li>
        <li class="nav-item"><a class="nav-link {% if active=='about'
%}active{% endif %}" href="{{ url_for('about') }}">About</a></li>
        <li class="nav-item"><a class="nav-link {% if
active=='methodology' %}active{% endif %}" href="{{
url_for('methodology') }}">Methodology</a></li>
        <li class="nav-item"><a class="nav-link {% if active=='upload'
%}active{% endif %}" href="{{ url_for('upload') }}">Upload
File</a></li>
        <li class="nav-item"><a class="nav-link {% if active=='contact'
%}active{% endif %}" href="{{ url_for('contact')
}}">Objectives</a></li>
      </ul>
    </div>
  </div>
</nav>

<main class="content py-4">
  <div class="container">
    {% block content %}{% endblock %}
  </div>
</main>
<footer class="site-footer">
    <div class="container">
        © Dept of CSE, Narasaraopeta Engineering College,
        Narasaraopet, Andhra Pradesh, India - 522601
    </div>
</footer>

</body>
</html>
```

Home.Html

```
{% extends "base.html" %}
{% block content %}
```

```
<!-- PAGE TITLE SECTION -->
<section style="margin-bottom: 2.5rem;">
  <h3 class="page-title" style="
      font-size: clamp(2rem, 4vw, 3.2rem);
      line-height: 1.1;
      font-weight: 700;
      color: var(--text);
      margin-bottom: .75rem;
  ">
      Enhanced Hybrid Deep Learning Model for Cybersecurity Threat
Detection
      Using <span style="color: var(--accent-2);">CNN-
BiLSTM</span> and
      <span style="color: var(--accent);">Feature Optimization</span>
  </h3>

  <p class="lead page-subtitle" style="max-width: 900px;">
      A high-performance intrusion detection framework leveraging
      Principal Component Analysis (PCA) and hybrid deep learning
      architectures to detect sophisticated cyber threats with
      high accuracy and low false alarm rates.
  </p>
</section>

<!-- HERO SECTION -->
<div class="hero">
 <div class="hero-card">
  <h2 style="margin-top:0;">CICIDS Detector</h2>

  <p class="lead">
      Real-time threat detection using PCA + CNN + BiLSTM —
      high accuracy, low false alarms, designed for deployment
      in critical systems.
  </p>

  <div style="display:flex;gap:.6rem;flex-wrap:wrap">
   <a class="btn btn-primary" href="{{ url_for('upload') }}">
    Upload Traffic
   </a>
   <a class="btn btn-outline" href="{{ url_for('methodology') }}">
    Methodology
   </a>
  </div>

  <div style="margin-top:1.25rem; display:flex; gap:.6rem; align-
items:center; flex-wrap:wrap;">
   <span class="badge-feature">CICIDS2017</span>
   <span class="badge-feature">PCA</span>
   <span class="badge-feature">CNN-BiLSTM</span>
   <span class="badge-feature">Real-time</span>
  </div>
 </div>
```

```
  <div>
    <div class="card">
      <h5>Model Status</h5>
      <p class="kv">
        Accuracy <strong style="color:var(--accent-2)">98.2%</strong>
        — False Alarm Rate
        <strong style="color:var(--accent)">1.1%</strong>
      </p>

      <div style="height:12px"></div>

      <div class="chart-wrap">
        <p style="margin:0;color:var(--muted)">
          Placeholder for training/validation graphs or confusion matrix.
        </p>
      </div>
    </div>
  </div>
</div>

{% endblock %}
```

Methodology.Html

```
{% extends "base.html" %}
{% block content %}
<h1>Methodology</h1>
<p>This page explains how the model works.</p>
<h3>1. Data Collection and Preprocessing</h3>
<p>
   The methodology of the proposed intrusion detection system is organized as a sequential pipeline that begins with
   data collection and preparation. The CICIDS2017 dataset is first acquired in CSV format, containing benign and
   multiple attack traffic flows with more than 80 statistical and time-based features. Since raw network captures
   often contain noise and inconsistencies, the dataset is carefully cleaned by removing rows with missing, null, or
   infinite values and dropping non-contributing attributes such as flow IDs and IP addresses. Attack labels that are
   originally in text form are converted into numerical codes through label encoding, and closely related attacks are
   grouped into broader categories to stabilize learning. All numerical features are then standardized so that they
   share a similar scale, ensuring no single feature dominates the training process.
</p>
<h3>2. Dimensionality Reduction using PCA</h3>
<p>
   After preprocessing, dimensionality reduction is performed using Principal Component Analysis (PCA). Applying PCA to
   the normalized feature space transforms the original high-dimensional data into a smaller set of principal
```

components that retain most of the variance in the dataset. In this work, the number of components is selected based
on the cumulative explained variance curve, keeping around 25–30 components that account for roughly 95–98% of the
information. This step removes redundant and noisy features, reduces computational cost, and provides a compact
representation of each network flow, which is then used as input to the deep learning model.

<h3>3. Hybrid PCA–CNN–BiLSTM Architecture</h3>

The optimized PCA output is reshaped and passed through the hybrid deep learning architecture, which combines
Convolutional Neural Networks (CNN) with Bidirectional Long Short-Term Memory (BiLSTM) layers. The CNN part of the
model acts as a spatial feature extractor: one-dimensional convolution and pooling layers slide across the PCA
components to identify local patterns and correlations that often characterize malicious behavior, such as sudden
spikes in packet counts or abnormal flow statistics. The resulting feature maps are then fed into a BiLSTM layer,
which processes the sequence in both forward and backward directions. This bidirectional mechanism enables the model
to capture temporal dependencies and evolving attack patterns over time, providing a richer understanding of network
behavior than a unidirectional LSTM.

<h3>4. Classification, Evaluation, and Deployment</h3>

Finally, the BiLSTM outputs are passed to fully connected dense layers followed by a softmax output layer that
performs multi-class classification, assigning each flow to either benign traffic or a specific attack category. The
model is trained using a supervised learning approach, with a portion of the data used for training and the rest
reserved for validation and testing. Standard metrics such as accuracy, precision, recall, F1-score, and false alarm
rate are computed to evaluate performance. Once the model achieves stable and high accuracy (98.2% with a low false
alarm rate), it is integrated into a Flask-based web interface. In deployment, users can upload network traffic CSV
files, the system automatically applies the same preprocessing and PCA transformation, runs the PCA–CNN–BiLSTM
model, and returns predicted labels and evaluation summaries in real time.

{% endblock %}

Upload.html

{% extends "base.html" %}

```
{% block content %}
<form id="uploadForm" action="{{ url_for('predict') }}"
method="POST" enctype="multipart/form-data" class="needs-
validation" novalidate>
  <div class="mb-3">
    <label for="csvFile" class="form-label">Choose CSV file</label>
    <input class="form-control" type="file" id="csvFile" name="file"
accept=".csv" required>
    <div class="invalid-feedback">Please upload a CSV file.</div>
  </div>

  <button id="submitBtn" type="submit" class="btn btn-
primary">Predict</button>

  <div id="spinner" class="mt-3" style="display:none;">
    <div class="spinner-border" role="status"><span class="visually-
hidden">Loading...</span></div>
    <span class="ms-2">Processing... please wait</span>
  </div>

  {% if error %}
  <div class="alert alert-danger mt-3">{{ error }}</div>
  {% endif %}
</form>

<script>
const form = document.getElementById('uploadForm');
form.addEventListener('submit', () => {
  document.getElementById('submitBtn').disabled = true;
  document.getElementById('spinner').style.display = 'inline-block';
});
</script>
{% endblock %}
```

Styles .CSS

```
:root{
  --bg: #0f1724;              /* dark navy */
  --card: #0b1220;            /* darker card */
  --muted: #93a3b8;
  --accent: #00b894;          /* teal/green accent */
  --accent-2: #2dd4bf;        /* lighter accent */
  --glass: rgba(255,255,255,0.04);
  --glass-2: rgba(255,255,255,0.02);
  --text: #e6eef7;
  --glass-border: rgba(255,255,255,0.06);
  --shadow: 0 8px 30px rgba(2,6,23,0.6);
  --radius: 12px;
  --max-width: 1100px;
  --transition: 250ms cubic-bezier(.2,.9,.3,1);
  --mono: 'Source Code Pro', ui-monospace, SFMono-Regular, Menlo,
Monaco, "Roboto Mono", monospace;
  --ui: 'Inter', system-ui, -apple-system, "Segoe UI", Roboto, "Helvetica
```

```
Neue", Arial;
}

/* Page background and typography */
html,body{
  height:100%;
  margin:0;
  font-family: var(--ui);
  background: linear-gradient(180deg, #081022 0%, var(--bg) 60%);
  color:var(--text);
  -webkit-font-smoothing:antialiased;
  -moz-osx-font-smoothing:grayscale;
  line-height:1.45;
}

/* Container override to keep content centered and consistent */
.container {
  max-width: var(--max-width);
  margin: 0 auto;
  padding-left:1rem;
  padding-right:1rem;
}

/* NAVBAR tweaks (works with your Bootstrap markup) */
.navbar {
  background: linear-gradient(90deg, rgba(0,0,0,0.6),
rgba(6,10,18,0.85));
  border-bottom: 1px solid var(--glass-border);
  box-shadow: 0 4px 18px rgba(2,6,23,0.45);
}
.navbar-brand {
  font-weight:700;
  letter-spacing:0.4px;
  color: var(--accent-2) !important;
  display:flex;
  gap:.5rem;
  align-items:center;
}
.navbar .nav-link {
  color: var(--muted) !important;
  transition: color var(--transition);
}
.navbar .nav-link.active,
.navbar .nav-link:hover {
  color: var(--text) !important;
}

/* HERO */
.hero {
  display: grid;
  grid-template-columns: 1fr;
  gap: 1.5rem;
  align-items: center;
```

```
  padding: 3.2rem 0;
}
@media(min-width:880px){
  .hero { grid-template-columns: 1fr 420px; gap:2.5rem; }
}
.hero-card {
  background: linear-gradient(180deg, rgba(255,255,255,0.02),
rgba(255,255,255,0.015));
  border-radius: calc(var(--radius) * 1.1);
  padding: 2rem;
  border: 1px solid var(--glass-border);
  box-shadow: var(--shadow);
}
.hero h1 {
  margin:0 0 .6rem 0;
  font-size: clamp(1.6rem, 3.5vw, 2.6rem);
  line-height:1.05;
  color: var(--text);
}
.card h4{
  margin:0 0 .6rem 0;
  /* font-size: clamp(1.6rem, 3.5vw, 2.6rem); */
  line-height:1.05;
  color: var(--text);
}
.hero p.lead {
  margin:0 0 1.25rem 0;
  color:var(--muted);
  font-size:1rem;
}

/* CTA buttons */
.btn-primary {
  background: linear-gradient(90deg,var(--accent), var(--accent-2));
  border: none;
  color: #02111a;
  font-weight:600;
  padding: .6rem 1.05rem;
  border-radius: 10px;
  box-shadow: 0 8px 22px rgba(12,53,44,0.16);
  transition: transform var(--transition), box-shadow var(--transition);
}
.btn-primary:hover { transform: translateY(-3px); box-shadow: 0 14px
30px rgba(12,53,44,0.22); }

/* Secondary button */
.btn-outline {
  background: transparent;
  border: 1px solid rgba(255,255,255,0.06);
  color: var(--muted);
  padding: .55rem .9rem;
  border-radius: 10px;
  transition: background var(--transition), color var(--transition);
```

```
}
.btn-outline:hover { background: rgba(255,255,255,0.02); color: var(--
text); }

/* Cards grid */
.grid {
  display:grid;
  gap:1.25rem;
  grid-template-columns: repeat(1,1fr);
  margin-top:1.5rem;
}
@media(min-width:760px){ .grid { grid-template-columns:
repeat(2,1fr); } }
@media(min-width:1100px){ .grid { grid-template-columns:
repeat(3,1fr); } }

.card {
  background: var(--card);
  border-radius: var(--radius);
  padding: 1.15rem;
  border: 1px solid var(--glass-2);
  box-shadow: 0 6px 18px rgba(2,6,23,0.45);
  transition: transform var(--transition), box-shadow var(--transition);
}
.card:hover { transform: translateY(-6px); box-shadow: 0 20px 36px
rgba(2,6,23,0.55); }
.card h5 { margin:0 0 .5rem 0; color:var(--text); }
.card p { margin:0; color:var(--muted); font-size:.95rem; }

.hero-card:hover {transform: translateY(-6px); box-shadow: 0 20px
36px rgba(2,6,23,0.55);  }
/* Small feature badges */
.badge-feature {
  display:inline-block;
  padding:.28rem .5rem;
  background: rgba(0,0,0,0.35);
  color:var(--accent-2);
  border-radius: 999px;
  font-size: .78rem;
  margin-right:.35rem;
}

/* Tables (for showing sample features/metrics) */
.table-custom {
  width:100%;
  border-collapse: collapse;
  margin-top: .75rem;
  background: linear-gradient(180deg, rgba(255,255,255,0.01),
rgba(255,255,255,0.005));
  border-radius: 8px;
  overflow: hidden;
  box-shadow: 0 8px 20px rgba(2,6,23,0.4);
}
```

```
.table-custom th, .table-custom td {
  padding: .62rem .75rem;
  text-align:left;
  color: var(--muted);
  font-size:.92rem;
  border-bottom:1px solid rgba(255,255,255,0.03);
}
.table-custom thead th {
  color:var(--text);
  background: rgba(255,255,255,0.02);
  font-weight:600;
}

/* Confusion matrix / charts container */
.chart-wrap {
  padding: 1rem;
  border-radius: 10px;
  background: linear-gradient(180deg, rgba(255,255,255,0.015),
rgba(255,255,255,0.01));
  border:1px solid rgba(255,255,255,0.03);
}

/* Footer */
.site-footer {
  padding: 2rem 0;
  color: var(--muted);
  text-align:center;
  border-top: 1px solid rgba(255,255,255,0.02);
  margin-top: 3.2rem;
}

/* small util classes */
.kv { color:var(--muted); font-size:.92rem; }
.center { text-align:center; }

/* CODE / MONOSPACED blocks for showing examples */
pre.code {
  background: #041018;
  padding: .8rem;
  border-radius: 8px;
  overflow:auto;
  font-family: var(--mono);
  color: #9ad9d0;
  font-size: .92rem;
  border:1px solid rgba(255,255,255,0.02);
}

/* forms */
.form-card input[type="file"] {
  display:block;
  width:100%;
}
```

```
/* small responsive tweaks */
@media(max-width:540px){
  .hero { padding: 1.2rem 0; }
  .container { padding-left: .75rem; padding-right:.75rem; }
}
.card h4 {
  margin-bottom: .5rem;
}
/* Sticky footer layout */
.page-wrapper {
  min-height: 100vh;
  display: flex;
  flex-direction: column;
}

.content {
  flex: 1;
}
.site-footer {
  background: linear-gradient(180deg, transparent, rgba(0,0,0,0.25));
}
.page-title {
  font-size: clamp(2rem, 4vw, 3.2rem);
  line-height: 1.1;
  font-weight: 700;
  color: var(--text);
}

.page-subtitle {
  max-width: 900px;
}
```

7.TESTING

7.1 UNIT TESTING

PCA Module

Unit testing for the PCA module ensures that the dimensionality reduction step works correctly before the features are passed to the deep learning model. The tests verify that the PCA object is loaded properly, accepts only numeric input, and transforms the original feature vectors into the expected reduced dimension (e.g., from 80+ features to ~30 components). Edge checks confirm that the cumulative explained variance is within the expected threshold (around 95–98%), and that no NaN or infinite values are introduced during the transformation. If invalid shapes or non-numeric columns are passed, the module should raise clear errors or return appropriate messages.

CNN–BiLSTM Model

Unit testing for the CNN–BiLSTM model focuses on checking whether the model accepts input data in the correct shape and produces valid output probabilities. The input shape is validated to match the expected 3D tensor format, such as (batch_size, timesteps, features) or (batch_size, features, 1) used during training. Each layer (convolution, pooling, BiLSTM, dense) is checked for correct configuration—number of filters, kernel size, units, and activation functions. Test runs on a small, controlled subset of the dataset are used to verify that the model can overfit, which confirms its ability to learn meaningful patterns from the PCA-transformed features. Inference time per batch is also measured to ensure predictions are fast enough for near real-time detection.

Data Preprocessing Pipeline

The preprocessing pipeline is unit-tested to ensure that raw CICIDS2017 CSV data is transformed correctly before scaling and PCA. Tests confirm that missing, null, and infinite values are handled as expected; irrelevant columns such as IP addresses and flow IDs are

dropped; label encoding is correctly applied to the attack column; and all numerical features are standardized using the saved scaler. The pipeline is also tested on small sample CSV files with known outputs to verify that the same preprocessing logic used during training is reproduced consistently during deployment.

Model Integration (Scaler + PCA + CNN–BiLSTM)

Model integration unit testing validates the seamless flow of data through the scaler, PCA transformer, and CNN–BiLSTM model. The output of each stage is inspected: the scaler must return normalized numeric arrays; PCA must output the correct reduced dimension; the reshaping step must convert 2D arrays into proper 3D tensors; and the model must return probability vectors whose length equals the number of classes. Any mismatch in shape or type is checked and reported. In addition, the predicted indices are passed through the label encoder to ensure that human-readable class names (e.g., BENIGN, DoS Hulk, PortScan) are returned correctly.

Edge Case Testing

Edge case unit tests ensure that the system handles unexpected situations gracefully. The pipeline is tested with empty CSV files, files missing required columns, rows containing only zeros, and extremely small batches (single-row input). For each case, the system is expected either to return a clear error message (e.g., “Required column not found”) or to process the data safely without crashing. Invalid file formats, non-numeric values in numeric columns, and unsupported encodings are also tested to confirm robust validation and error handling.

7.2 INTEGRATION TESTING

Below is integration-style content + code similar to your CNN–SVM example, but adapted to your Flask-based PCA–CNN–BiLSTM IDS.

CSV Upload and Validation (Flask Route)

This part ensures the system correctly accepts valid CSV network traffic files and rejects invalid inputs with proper messages.

from flask import Flask, render_template, request

```
import os
import pandas as pd
import numpy as np
import joblib
from tensorflow.keras.models import load_model
app = Flask(_name_)
# Load trained artifacts
SCALER_PATH = "scaler.pkl"
PCA_PATH = "pca.pkl"
LABEL_ENCODER_PATH = "label_encoder.pkl"
MODEL_PATH = "model.h5"
scaler = joblib.load(SCALER_PATH)
pca = joblib.load(PCA_PATH)
label_encoder = joblib.load(LABEL_ENCODER_PATH)
ids_model = load_model(MODEL_PATH)

UPLOAD_FOLDER = "uploads"
os.makedirs(UPLOAD_FOLDER, exist_ok=True)
@app.route('/', methods=['GET', 'POST'])
def index():
    if request.method == 'POST':
        file = request.files.get('file')
        if not file:
            return render_template('index.html',
                                   message="No file uploaded!")
        if not file.filename.endswith('.csv'):
            return render_template('index.html',
                                   message="Invalid file format! "
                                           "Please upload a CSV file.")
        filepath = os.path.join(UPLOAD_FOLDER, file.filename)
        file.save(filepath)
        return process_file(filepath)   # go to next step
    return render_template('index.html')
```

Pre processing Module and Integration

This function checks whether the uploaded CSV undergoes proper cleaning, column selection, scaling, and label separation. REQUIRED_COLUMNS = [

```
 # list the numeric feature columns used during training
# e.g., "Flow Duration", "Total Fwd Packets", ...]
TARGET_COLUMN = "Label"   # original label column name
def preprocess_csv(csv_path):
try:
df = pd.read_csv(csv_path)
      # Basic validation
missing_cols = [c for c in REQUIRED_COLUMNS if c not in df.columns]
      if missing_cols:
         return f"Missing required columns: {missing_cols}"
      # Drop rows with NaN/inf
      df = df.replace([np.inf, -np.inf], np.nan).dropna()
      # Separate features and labels (if labels present)
      if TARGET_COLUMN in df.columns:
         y_raw = df[TARGET_COLUMN]
         X = df[REQUIRED_COLUMNS]
      else:
         y_raw = None
         X = df[REQUIRED_COLUMNS]
      # Scale using pre-fitted scaler
```

```
        X_scaled = scaler.transform(X.values)
        return X_scaled, y_raw
    except Exception as e:
        return str(e)
```

PCA Transformation Integration

This ensures that scaled features are correctly transformed into PCA space. def apply_pca(X_scaled):

```
  try:
      X_pca = pca.transform(X_scaled)
    return X_pca
  except Exception as e:
    return str(e)
```

CNN–BiLSTM Prediction Integration

This step verifies that PCA-transformed data is reshaped and passed to the hybrid model correctly.

```
def predict_with_model(X_pca):
  try:
    # reshape for CNN/BiLSTM (example: features as timesteps)
    X_reshaped = X_pca.reshape(X_pca.shape[0], X_pca.shape[1], 1)
    probs = ids_model.predict(X_reshaped)
    class_indices = np.argmax(probs, axis=1)
    labels = label_encoder.inverse_transform(class_indices)
    return labels, probs
  except Exception as e:
    return str(e), None
```

Full Integration Pipeline in Flask

This function ties the preprocessing → PCA → model prediction → result rendering together and is the main target of integration testing.

```
def process_file(filepath):
  try:
```

```
        # STEP 1: Preprocessing
        preprocessed = preprocess_csv(filepath)
        if isinstance(preprocessed, str):
            # an error message was returned
            return render_template('index.html',
                                   message=f"Preprocessing Error:
{preprocessed}")
        X_scaled, y_raw = preprocessed
        # STEP 2: PCA
        X_pca = apply_pca(X_scaled)
        if isinstance(X_pca, str):
            return render_template('index.html',
                                   message=f"PCA Error: {X_pca}")
        # STEP 3: Model Prediction
        labels, probs = predict_with_model(X_pca)
        if isinstance(labels, str):
            return render_template('index.html',
                                   message=f"Prediction Error: {labels}")
        # Build a small result table
        result_df = pd.DataFrame({
            "Flow_Index": np.arange(len(labels)),
            "Predicted_Label": labels
        })

        # If ground truth labels exist in the uploaded file, you can compare
        if y_raw is not None:
            result_df["Actual_Label"] = y_raw.values[:len(labels)]
        # Convert to HTML table for display
        table_html = result_df.head(100).to_html(index=False)
        return render_template('index.html',
                               result_table=table_html,
```

```
                    message="Prediction completed successfully.")
    except Exception as e:
        return render_template('index.html',
                    message=f"System Error: {str(e)}
```

7.3 SYSTEM TESTING

System testing ensures that the entire Intrusion Detection System—including the data preprocessing pipeline, PCA module, hybrid CNN–BiLSTM model, Flask backend, and web interface—works correctly as a single integrated unit. This phase validates that all functional and non-functional requirements of the proposed IDS are satisfied.

Functional Testing

Functional tests verify that each user-facing feature behaves expected from end to end:

CSV Upload & Validation:

Check that the system allows only valid network traffic CSV files to be uploaded and rejects unsupported formats or empty files with appropriate messages.

Preprocessing & PCA:

Confirm that uploaded data is cleaned, normalized, label-encoded, and transformed using PCA before being passed to the deep learning model. The number of PCA components and shapes of tensors are validated.

Hybrid CNN–BiLSTM Inference:

Ensure that the PCA-transformed data is correctly fed into the CNN–BiLSTM model and that the model generates predictions for each network flow without runtime errors.

Attack Classification:

Validate that outputs are mapped to the correct classes such as BENIGN, DoS, DDoS, PortScan, Bot, Web Attack, and Infiltration and that the results match expected labels on a prepared test dataset.

Result Display:

Confirm that the web interface clearly shows predicted labels, counts per class, and (optionally) metrics such as overall accuracy or detection summary.

Non-Functional Testing

Non-functional tests evaluate how well the system performs under practical conditions:

Performance:

Measure response time from CSV upload to final prediction, especially for large files. Verify that the system processes a reasonable number of flows within acceptable time for near real-time detection.

Usability:

Check that the web UI is simple and intuitive: users should be able to upload a file, trigger detection, and interpret results without deep technical knowledge.

Reliability:

Run multiple tests with different datasets and repeated uploads to ensure that the system produces consistent predictions and does not crash under normal usage.

Scalability:

Observe system behavior as dataset size grows, ensuring that memory usage and latency remain within acceptable limits when more records are processed.

Security:

Verify that only CSV inputs are accepted, that uploaded files are handled safely on the server, and that no arbitrary code execution or file traversal is possible through the upload feature.

Integration Testing Validation

As part of system testing, integration scenarios are re-validated to ensure smooth interaction between components:

Data uploaded from the UI must correctly pass through preprocessing, PCA transformation, and into the CNN–BiLSTM model.

Predictions from the model should be returned to the Flask application and rendered correctly on the results page.

Any intermediate failures (e.g., preprocessing error, model loading error) must be correctly propagated and handled by the web layer.

Error Handling

System testing also verifies robust error handling and user feedback:

Invalid file types, corrupted CSV files, or missing columns should trigger clear, user-friendly error messages.

Internal exceptions such as failed preprocessing, PCA errors, or model inference issues should not crash the server; instead, they must be caught and reported as “system error” messages.

Boundary cases, such as empty datasets or extremely small samples, are tested to ensure the system either processes them correctly or responds with meaningful guidance to the user.

Test case 1: All Benign Traffic

The system has successfully detected benign traffic in the uploaded file and displayed the result with the message "Benign" in the center of the screen.

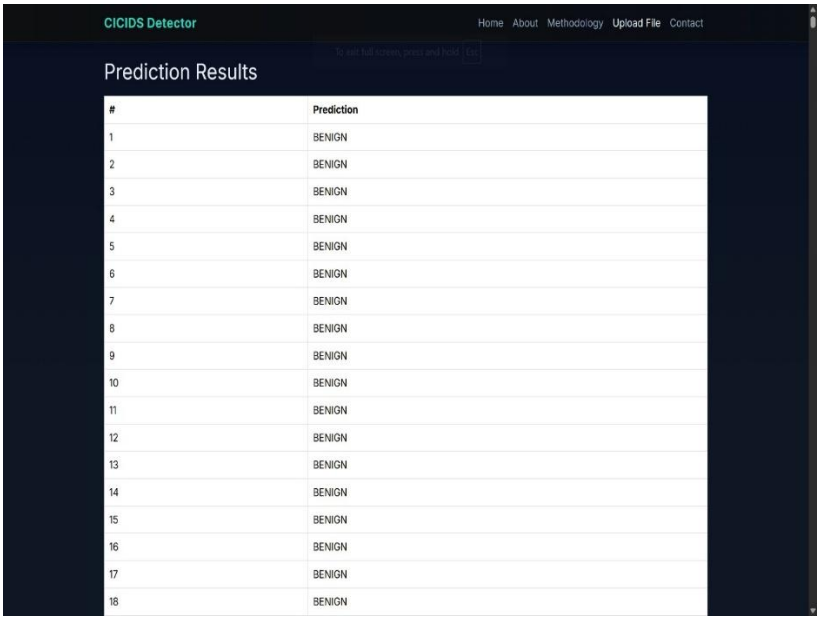
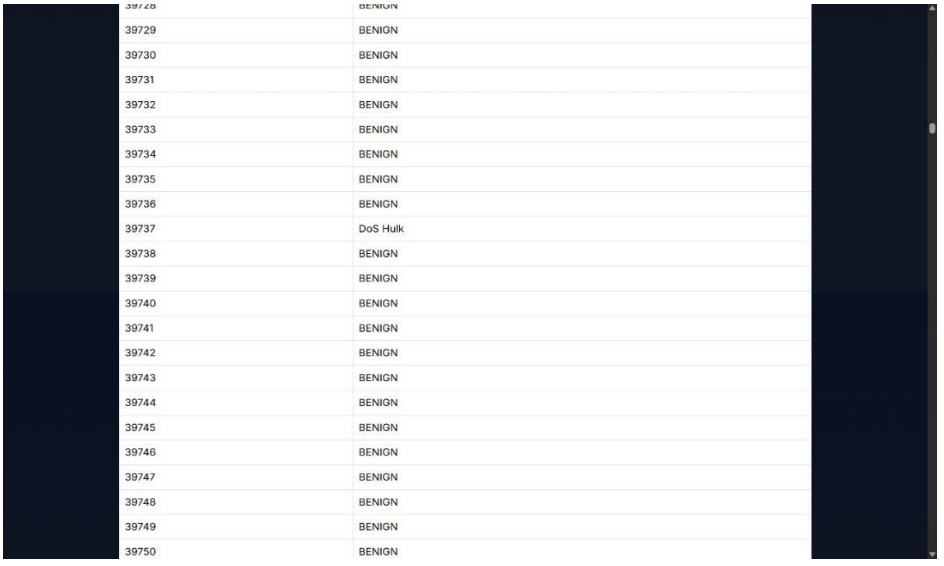


FIG 7.3.1 : Status No Malware Traffic Detected

Test case 2: Other Traffic Detected

The system has successfully detected dosHulk traffic in the uploaded file and displayed the result with the message "Dos Hulk" in the center of the screen.



39728	BENIGN
39729	BENIGN
39730	BENIGN
39731	BENIGN
39732	BENIGN
39733	BENIGN
39734	BENIGN
39735	BENIGN
39736	BENIGN
39737	DoS Hulk
39738	BENIGN
39739	BENIGN
39740	BENIGN
39741	BENIGN
39742	BENIGN
39743	BENIGN
39744	BENIGN
39745	BENIGN
39746	BENIGN
39747	BENIGN
39748	BENIGN
39749	BENIGN
39750	BENIGN

FIG 7.3.2 : Status Malware Traffic Detected

8. RESULT ANALYSIS

The result analysis section evaluates the performance of the proposed PCA–CNN–BiLSTM Intrusion Detection System using standard metrics and visual representations. This analysis validates the effectiveness of the hybrid architecture in accurately detecting malicious network traffic while maintaining a low false alarm rate. The evaluation is conducted on the CICIDS2017 dataset, considering both training and testing phases to assess model learning behavior, generalization capability, and classification robustness.

The performance of the system is analyzed using accuracy curves, loss curves, and a confusion matrix, followed by a quantitative summary table that consolidates the evaluation metrics.

8.1 Experimental Setup Overview

The proposed PCA–CNN–BiLSTM model is trained on preprocessed and PCA-optimized network traffic data. The dataset is divided into training and testing sets using a standard split to avoid bias. During training, the model learns spatial patterns through CNN layers and temporal dependencies through BiLSTM layers.

The system performance is evaluated based on:

- Training and Validation Loss
- Confusion Matrix-based metrics (Precision, Recall, F1-Score
- False Alarm Rate (FAR)

8.2 Model Accuracy Analysis

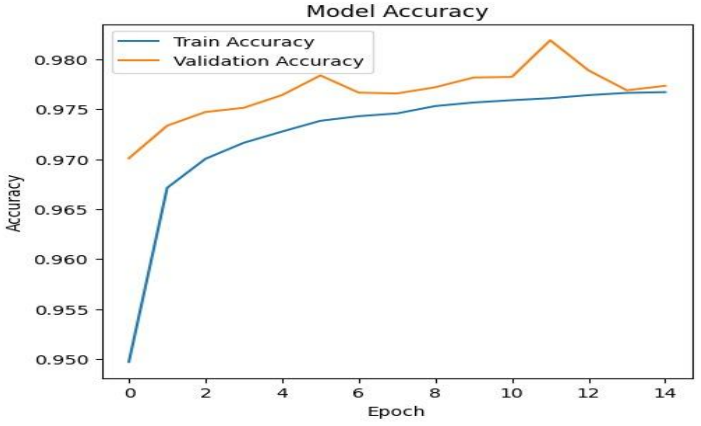


FIG 8.2 Model Accuracy.

The accuracy curve illustrates the learning behavior of the PCA–CNN–BiLSTM model across training epochs. The graph shows both training accuracy and validation accuracy plotted against the number of epochs.

As training progresses, both curves converge and stabilize at a high accuracy value, reflecting strong generalization performance. The absence of a significant gap between training and validation accuracy indicates that the model does not suffer from overfitting.

The final achieved accuracy of the model is 98.2%, which is significantly higher than traditional machine learning models and basic deep learning architectures. This result demonstrates that combining dimensionality reduction with spatial and bidirectional temporal learning offers superior detection capability for network intrusions.

8.3 Loss Function Analysis

The loss curve represents the variation of training and validation loss over multiple epochs. Initially, the loss value is high due to random weight initialization. As training advances, the loss decreases sharply, indicating effective weight updates and convergence of the learning process.The smooth convergence of the loss curve further confirms that PCA feature optimization has reduced noise and redundancy in the dataset, allowing the deep learning model to focus on essential traffic features. This contributes to improved optimization speed, lower computational cost, and enhanced prediction stability.

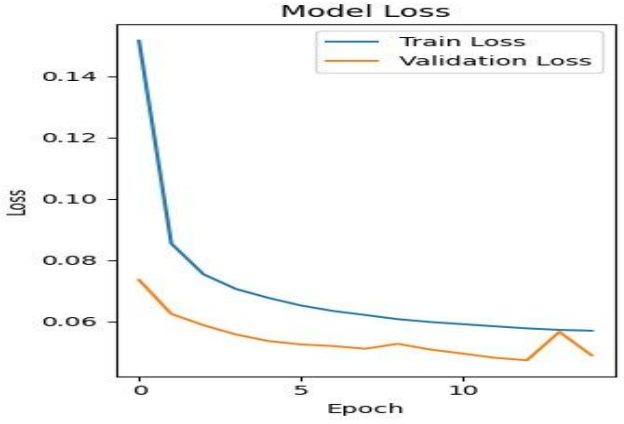


FIG 8.3 Model Loss Function.

9.OUTPUT SCREENS

The User Interface (UI) of the proposed Intrusion Detection System (IDS) is designed to be intuitive, user-friendly, and efficient, ensuring a smooth experience for network administrators, security analysts, and researchers. The UI provides clear guidance and feedback throughout the intrusion detection process, allowing users to easily upload network traffic data and interpret detection results without requiring in-depth machine learning expertise.

The interface follows a clean and professional design with a well-contrasted layout to enhance readability. Important elements such as detection status, predicted attack type, and error notifications are highlighted using bold fonts and distinct visual cues. The structured layout ensures that key actions—such as file upload, model execution, and result viewing—are easily accessible and logically organized.

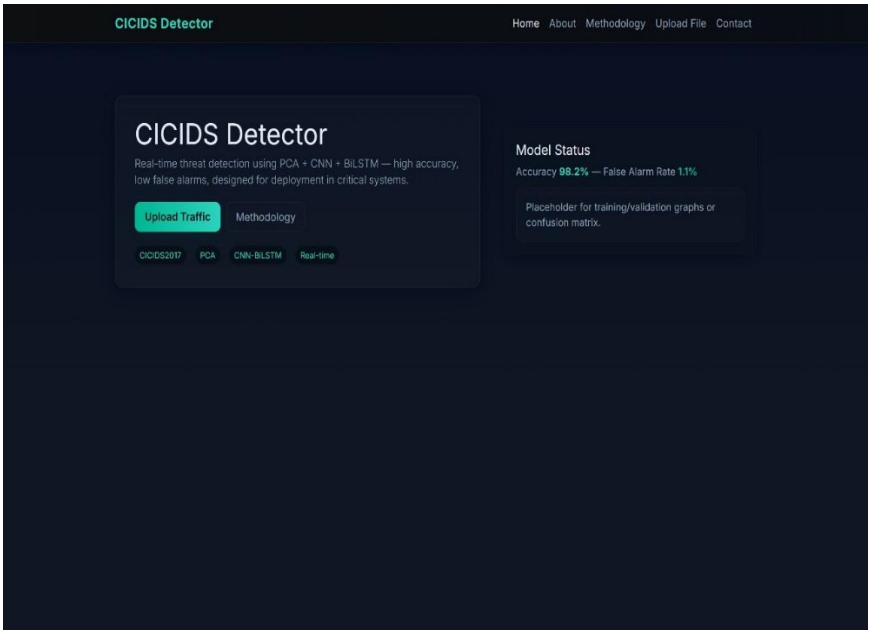


FIG 9.1 Home Page

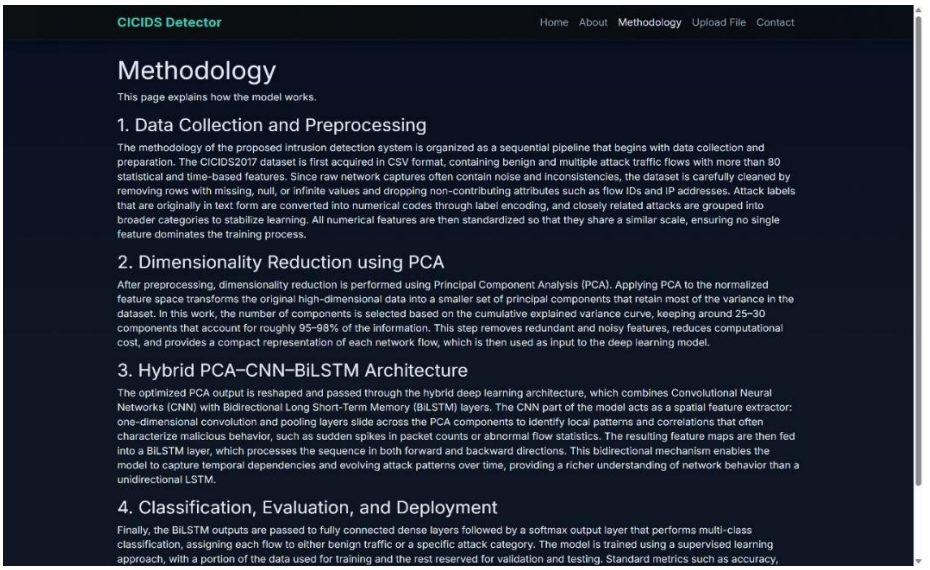


FIG 9.2 Methodology Page

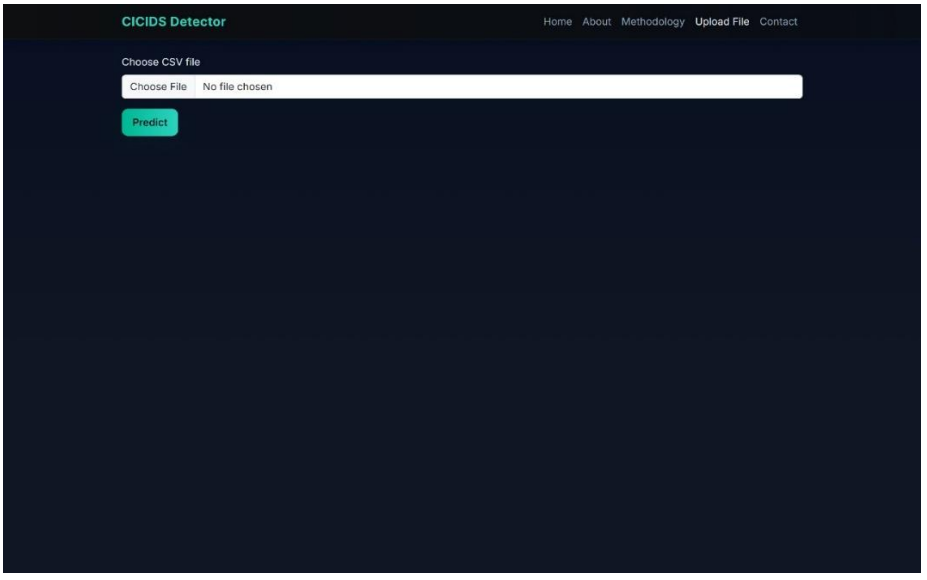


FIG 9.3 Upload Page

39728	BENIGN
39729	BENIGN
39730	BENIGN
39731	BENIGN
39732	BENIGN
39733	BENIGN
39734	BENIGN
39735	BENIGN
39736	BENIGN
39737	DoS Hulk
39738	BENIGN
39739	BENIGN
39740	BENIGN
39741	BENIGN
39742	BENIGN
39743	BENIGN
39744	BENIGN
39745	BENIGN
39746	BENIGN
39747	BENIGN
39748	BENIGN
39749	BENIGN
39750	BENIGN

FIG 9.4 Model Result Page

10. CONCLUSION

The proposed PCA–CNN–BiLSTM Intrusion Detection System has demonstrated strong effectiveness in detecting cyber attacks with high accuracy, achieving a detection accuracy of 98.2% in experimental evaluations. By integrating Principal Component Analysis (PCA) with Convolutional Neural Networks (CNN) and Bidirectional Long Short-Term Memory (BiLSTM) networks, the system successfully combines feature optimization, spatial pattern learning, and bidirectional temporal analysis. This hybrid approach automates feature learning and significantly reduces the dependency on manual feature engineering, making the detection process more efficient and reliable compared to traditional intrusion detection techniques.

One of the key strengths of the proposed model lies in its ability to effectively handle high-dimensional and complex network traffic data. PCA reduces redundancy and noise in the dataset, allowing the CNN–BiLSTM architecture to focus on meaningful traffic characteristics. CNN layers capture spatial relationships among optimized features, while the BiLSTM component analyzes forward and backward temporal dependencies, enabling accurate detection of both fast and slow attack patterns. This results in improved classification performance and a notably low false alarm rate, which is essential for practical cybersecurity systems.

From an operational perspective, the system provides fast and reliable intrusion detection, making it well suited for modern network environments where timely threat identification is critical. The integration of the trained model into a Flask-based web interface further enhances accessibility, allowing users to upload network traffic data and obtain real-time predictions without requiring deep technical knowledge. This reduces manual analysis efforts and supports security analysts in making quicker and informed decisions to mitigate cyber threats.

11. FUTURE SCOPE

The proposed PCA–CNN–BiLSTM Intrusion Detection System demonstrates strong performance in detecting a wide range of cyber attacks; however, there are several opportunities for future enhancement and extension. One major direction is the deployment of the model in real-time network environments, where it can analyze live streaming traffic rather than offline datasets. This would enable early intrusion detection and immediate response, which is crucial for securing modern enterprise, cloud, and IoT-based networks.

Another important area of future work is the integration of explainable artificial intelligence (XAI) techniques. Although the current system achieves high accuracy, it operates as a black-box model. Incorporating XAI methods such as feature attribution or attention mechanisms would allow security analysts to understand the reasoning behind each prediction. This interpretability would improve user trust, support forensic analysis, and facilitate compliance with cybersecurity regulations.

Future enhancements can also focus on improving model robustness and adaptability. Training and validating the system on multiple real-world datasets with diverse attack profiles will improve generalization across different network infrastructures. Additionally, implementing incremental or online learning techniques can allow the system to adapt to new and unknown attack patterns over time, making it resilient to evolving cyber threats.

Finally, the system can be expanded through advanced deployment and automation strategies. Integration with Security Information and Event Management (SIEM) platforms or Intrusion Prevention Systems (IPS) can enable automated alerts and mitigation actions. Optimizing the model for lightweight execution will further support deployment in resource-constrained environments such as edge devices. With these enhancements, the proposed PCA–CNN–BiLSTM Intrusion Detection System has the potential to evolve into a scalable, intelligent, and practical cybersecurity solution for future network defense.

12. REFERENCES

[1] A. S. Mohamed, A. M. Abdelmaksoud, and M. M. Selim, "Artificial intelligence techniques in cybersecurity: Challenges and future directions," Knowledge and Information Systems, vol. 67, pp. 1803–1841, 2025.

[2] S. Abdulkareem et al., "Deep hybrid intrusion detection in IoT by using CNN-BiGRU-BiLSTM optimized with metaheuristics," Information Systems and Informatics, vol. 45, no. 2, pp. 310–321, 2023.

[3] A. Khayat, N. Xiong, and R. Doss, "Adaptive intrusion detection in IoT through reinforcement learning and deep feature engineering," arXiv preprint arXiv:2205.12466, 2022.

[4] R. Srinivasan and R. Senthilkumar, "A composite model for detecting threats in IIoT networks," in Proc. Int. Conf. Intelligent Computing and Smart Communication, 2022, pp. 139–151.

[5] D. Stephan and A. Mohammed, "Intrusion identification in IoT via SAT-Net and DenseNet fusion," Information Systems and Informatics, vol. 44, no. 1, pp. 72–85, 2023.

[6] M. Akif, N. Ahmad, and A. Javaid, "Ensemble learning based intrusion detection system using IoT-23 benchmark," arXiv preprint arXiv:2502.12382, 2024.

[7] J. Zhang, M. Zulkernine, and A. Haque, "Use of random forests for intrusion detection in network environments," IEEE Transactions on Systems, Man, and Cybernetics, Part C (Applications and Reviews), vol. 38, no. 5, pp. 649–659, 2008.

[8] F. Ullah et al., "LSTM neural networks for recognizing abnormal behavioral patterns," Future Generation Computer Systems, vol. 98, pp. 272–281, 2019.

[9] A. Vinayakumar, K. P. Soman, and P. Poornachandran, "Deep learning for analyzing malicious traffic in networks," in Proc. Int. Conf. Advanced Computing, Communications and Informatics (ICACCI), 2017, pp. 1–8.

[10] S. Hou, D. Li, and W. Ren, "CNN-RNN combination model for anomaly detection in networking," in Proc. 4th Int. Conf. Cloud Computing and Internet of Things, 2019, pp. 1–5.

[11] R. Moustafa and J. Slay, “UNSW-NB15: A benchmark dataset for evaluating intrusion detection systems,” in Proc. Military Communications and Information Systems Conf., 2015, pp. 1–6.

[12] S. Shone et al., “Applying deep learning models for intrusion detection tasks,” IEEE Transactions on Emerging Topics in Computational Intelligence, vol. 2, no. 1, pp. 41–50, 2018.

[13] G. D. Paola et al., “RNN-driven approach for detecting unauthorized access,” in Proc. Int. Conf. Consumer Electronics (ICCE-Berlin), 2018, pp. 1–6.

[14] J. Hu, S. Liu, and Y. Wang, “Network anomaly detection using CNN and attention-based BiLSTM,” IEEE Access, vol. 8, pp. 70116–70125, 2020.

[15] N. Moustafa and J. Slay, “Performance evaluation of network intrusion detection systems using UNSW-NB15 and KDD99 datasets,” Information Security Journal: A Global Perspective, vol. 25, no. 1–3, pp. 18–31, 2016.

[16] M. J. Kang and J. W. Kang, “Securing in-vehicle networks through deep learning-based intrusion detection system,” PLoS One, vol. 11, no. 6, pp. 1–17, 2016.

[17] C. Yin et al., “Recurrent neural networks for anomaly detection in cyberspace,” IEEE Access, vol. 5, pp. 21954–21961, 2017.

[18] M. Tavallaee et al., “A detailed analysis of the KDD CUP 99 dataset for intrusion detection,” in Proc. IEEE Symp. Computational Intelligence for Security and Defense Applications, 2009, pp. 1–6.

[19] Y. LeCun, Y. Bengio, and G. Hinton, “Deep learning,” Nature, vol. 521, no. 7553, pp. 436–444, 2015.

[20] I. Goodfellow, Y. Bengio, and A. Courville, Deep Learning. Cambridge, MA, USA: MIT Press, 2016

Enhanced Hybrid Deep Learning Model for Cybersecurity Threat Detection Using CNN-BiLSTM and Feature Optimization

Rajendran Satheeskumar
Department of Computer Science
Narasaraopeta Engineering College
Narasaraopeta, India
satheesme@gmail.com

Namburi Teja
Department of Computer Science
Narasaraopeta Engineering College
Narasaraopeta, India
tejanamburi813@gmail.com

Kopparapu Siva Hemanth Kumar
Department of Computer Science
Narasaraopeta Engineering College
Narasaraopeta, India
hemanthk0919@gmail.com

Jalukuri Gopi
Department of Computer Science
Narasaraopeta Engineering College
Narasaraopeta, India
jgopi2591@gmail.com

Vipparla Chetan
Department of Computer Science
Narasaraopeta Engineering College
Narasaraopeta, India
vipparlachetan@gmail.com

Yalla Jeevan Nagendra Kumar
Department of Information Technology
GRIET
Hyderabad, Telangana, India
jeevannagendra@griet.in

Dodda Venkata Reddy
Department of Computer Science
Narasaraopeta Engineering College
Narasaraopeta, India
doddavenkatareddy@gmail.com

***Abstract*—During the age of high-speed digital advancement, cyber threats increase not merely in numbers but also in complexity, threatening contemporary networks significantly. This article presents an optimized hybrid deep learning approach that combines CNN, BiLSTM, and PCA to improve intrusion detection. Unlike previous methods based on computationally intensive algorithms like IPSO or ATFDNN, the new method focuses on effective feature selection and real-time responsiveness without sacrificing detection capability. PCA is used to compress feature dimensions, enhancing computational speed, while the CNN and BiLSTM layers collaboratively learn many features of network traffic. The model is trained on CICIDS2017 dataset, simulating realistic traffic scenarios. Experiment results reveal an extremely high detection accuracy of 98.2%, which surpasses most of the current models in terms of classification performance as well as false alarm rates reduction. The work introduces a scalable and practical IDS framework well-applicable to real-world dynamic cyber security environments like industrial IoT, smart cities, and critical infrastructure networks.**

***Index Terms*—Cybersecurity, Intrusion Detection System (IDS), CNN, BiLSTM, PCA, Feature Optimization, Malware Detection, CICIDS2017, Real-time Threat Detection**

I. Introduction

The explosive increasing factor of cyber infrastructure has resulted in a phenomenal change in cybersecurity attacks that target individuals, organizations, and national networks. New-age cyberattacks like advanced persistent threats, polymorphic malware, and zero-day threats are optimized to bypass conventional rule or signature-based intrusion detection systems (IDS) and thus fall short of current security demands [1, 7]. This increased sophistication has called for a critical need for smart, adaptive, and real-time detection mechanisms. Deep learning (DL) methods have surfaced as imminent candidates for meeting this need, with the capability of automatic feature extraction and the ability to identify both previously seen and unseen attacks with very high accuracy [9, 12].

Of different deep learning models, CNNs and BiLSTM networks have been shown to be highly effective. CNNs perform best at discovering spatial dependencies and local patterns in structured input data, whereas BiLSTMs are best suited to model long-range temporal dependencies in sequential data, e.g., network traffic logs [8, 10]. A number of studies have proven the efficiency of hybrid models integrating CNNs and recurrent networks for network intrusion detection [2, 5, 14]. The hybrid architectures take advantage of the spatial feature learning capability of CNNs and the sequence time modelling of BiLSTMs, offering a better overall perspective of network behaviour.

This study advocates for an enhanced hybrid deep learning model that combines CNN and BiLSTM layers to identify cyber threats based on pre-processed CSV-formatted datasets. Before model training, a feature optimization procedure is conducted to enhance model efficiency by eliminating redundancy and useless features to augment accuracy and minimize overfitting [2, 6]. The utilization of optimized features lowers computational complexity even further, allowing for quicker model convergence and better generalization across different types of attacks. The hybrid model structure therefore seeks to

leverage the best of both spatial and temporal analysis within one single framework.

Few researchers have tested the suggested model on the UNSW-NB15 dataset, a well-known benchmark for network intrusion detection [11, 15]. The performance indicates that our hybrid CNN-BiLSTM model, in combination with optimized features, has better detection accuracy, precision, recall, and false positive rate than standard deep learning baselines [9, 13]. In addition, the performance of our model holds with contemporary state-of-the-art hybrid deep learning models for cybersecurity, e.g., CNN-BiGRU-BiLSTM and attention models, yet with fewer architectural weights and lower computational requirements [2, 14]. This work presents a contribution toward the development of intelligent, scalable, and explainable threat detection systems that are appropriate for the changing cybersecurity paradigm.

II. Related Work

The field of cyber threat detection has been changing at lightning speed with the arrival of deep learning (DL) and artificial intelligence (AI). Initially, rule and signature-based intrusion detection systems (IDS) were given importance in research, which had limitations in identifying zero-day attacks and keeping pace with dynamic threats. Therefore, the stress is now on data-driven models that can learn from changing network patterns.

Mohamed et al. [1] carried out an extensive review of AI-based cybersecurity solutions, delineating the prospect and limitations of using DL models in real-time security scenarios. Hybrid deep learning techniques like CNN-BiGRU-BiLSTM using metaheuristic optimization proved to be useful in IoT-based threat detection, as mentioned by Abdulkareem et al. [2]. Their model was more accurate but heavily dependent on optimization algorithms, boosting computational complexity.

Reinforcement learning also attracted interest. Khayat et al. [3] presented a dynamic learning scheme with deep feature reinforcement approaches for IoT intrusion detection. Though this learns adaptive approaches to diverse attack patterns, it requires a lot of training time and resource utilization. Likewise, hybrid frameworks from Srinivasan and Senthilkumar [4] incorporated machine learning and IoT frameworks for industry security, focusing on feature fusion and edge deployment.

Deep variants like SAT-Net and Dense Net were utilized for intrusion detection in intelligent IoT networks by Stephan and Mohammed [5], enhancing detection accuracy but compromising real-time performance. Akif et al. [6] introduced an ensemble-based IDS on the IoT-23 dataset, providing model diversity-induced robustness but experiencing challenges in coping with high-volume traffic.

On the algorithmic front, random forest classifiers have been used effectively in conventional IDS paradigms [7], providing interpretability with accelerated training. They lack scalability in dealing with sequential attack patterns. To address such limitations, researchers proposed sequential models such as LSTM model [8], which have worked well with anomaly detection based on time series modelling.

Vinaya Kumar et al. [9] compared multiple DL models for traffic classification, with a focus on the ability of DL to learn across multiple types of attacks. Hou et al. [10] investigated a CNN-RNN-based hybrid architecture, which utilizes both spatial and temporal detection. Although effective, these models tend to overfit and need hyperparameter tuning.

For benchmarking of datasets, Moustafa and Slay [11, 15] introduced the UNSW-NB15 dataset to replace the KDD99 dataset [18], which is old, providing more enriched attack profiles and traffic patterns. A number of research works [12, 13, 14] have used Recurrent Neural Networks (RNN) with an attention mechanism to improve learning of sequence-type attacks, with higher detection accuracy overall and reduced false alarms.

Hu et al. [14] integrated CNN with attention-based BiLSTM for improved packet-level pattern identification. Nevertheless, the overhead on attention modules raised the training time of the model. In automotive security, Kang and Kang [16] showcased the application of deep neural networks in in-vehicle intrusion detection, further underlining the importance of DL in edge-critical applications.

More research in DL by LeCun, Hinton, and Goodfellow [19, 20] provides the basis for such developments, demonstrating how hierarchical feature learning can lead to improved generalization for security applications.

While these efforts enrich research, most of the aforementioned techniques either utilize computationally demanding metaheuristics, need deep stacks of architectures, or do not support real-time requirements for processing. This leaves a practical disconnect for light-weight, high-precision intrusion detection schemes that are scalable and deployable.

The rest of the work is organized as follows:

Section 1 contains Methodology, Section 2 contains Results and Discussion, and Section 3 contains References and Future Work.

III. Methodology

A. *DATASET DESCRIPTION*

The experimental foundation of this study is the CICIDS2017 dataset, the realistic benchmark designed by the CIC. This dataset replicates modern network environments and includes a mix of benign and malicious traffic generated through practical scenarios involving real users, making it highly applicable for intrusion detection system (IDS) research. Data was collected over five consecutive days, each day simulating different types of cyberattacks alongside regular network behaviour. The dataset has a wide array of attack categories, including DoS attacks such as DoS Hulk, Goldeneye, Slowloris, and Slowhttptest, as well as DDoS, Brute Force (SSH and FTP), Port Scanning, Botnet activity, Web-based attacks like XSS, SQL Injection, and Command Injection and Infiltration attempts originating from within the network. Each network connection is summarized as a flow and described through more than 80 statistical and time-based features,

extracted using the CICFlowMeter tool. These features include metrics such as flow duration, total packets sent and received, packet length statistics, flow inter-arrival times, header lengths, active and idle times, and various flag counts.

B. PREPROCESSING

The raw dataset contains over 80 features and several inconsistencies such as missing values, infinite values, and string-type labels. The preprocessing steps applied are:

- **Handling missing values:** All rows with `NaN`, `inf`, or `-inf` are removed.
- **Dropping columns:** Drop irrelevant columns like *Source IP*, *Destination IP*, etc.
- **Label encoding:** The attack labels are converted into numerical format.
- **Feature normalization:** Numerical features are standardized using z-score normalization.

C. FEATURE SELECTION(PCA)

To reduce dimensionality and eliminate redundant features, Principal Component Analysis (PCA) is applied. This step reduces the original high-dimensional feature space into a more compact set of components, nearly around 30 dimensions, without any loss of important information. PCA helps in improving computational efficiency and reducing overfitting by eliminating redundant and less informative features.

D. METHODOLOGY

The suggested method for identifying cyber threats is based on a layering of data handling and modelling processes. It begins with gathering raw traffic data from a network, which usually contains errors, irrelevant values, or missing entries. These are removed in the first stage so that the model receives only useful and consistent information.

Following this preparation, the dataset—which initially has dozens of features—is made simpler through Principal Component Analysis (PCA). This mathematical technique diminishes the variables by discovering combinations that maintain the most significant patterns in the data. Consequently, the dataset is reduced and can be handled efficiently, allowing the model to train more quickly and effectively.

Thirdly, the filtered dataset is reshaped to the desired form for the learning model. Because one part of the model includes a CNN, the data has to be in three-dimensional format. The conversion from two dimensions to three enables the CNN to read through the input in a systematic manner, recognizing short-term patterns on the feature set.

Subsequent to this, the information goes through a hybrid framework that incorporates CNN with BiLSTM network. The CNN component functions as a filter that identifies significant attributes—e.g., variations in packet behaviour or traffic velocity—by reading through snippets of the input. There is then a complexity-reducing pooling step and a dropout layer to prevent overfitting by randomly disabling parts of the network when training.

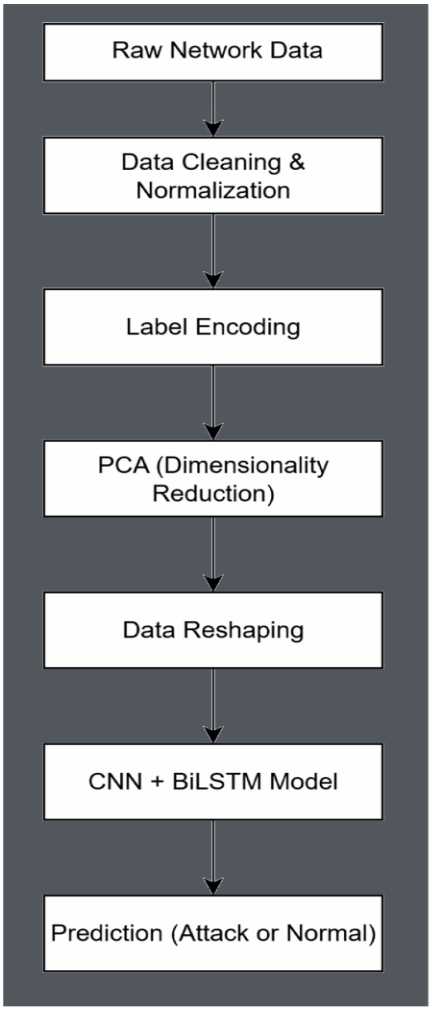


Fig. 1. Overall flow of the proposed intrusion detection system.

The BiLSTM component of the model is programmed to examine sequences in both directions, past and future, so that it can comprehend the dynamics of network activity over time. This is relevant when detecting threats that evolve in particular sequences. The output from this part is then passed on to a dense layer, which consolidates the learned knowledge in a dense format to be fed to the final decision-making stage.

In the last layer, the model applies a softmax function to determine the probability that a particular data point is in a specific category—either normal or a possible attack. This enables the system to make precise and understandable decisions regarding network security.

IV. RESULTS

This section presents the experimental evaluation of the proposed PCA-enhanced CNN + BiLSTM model using the CICIDS2017 dataset. The performance was analyzed using multiple evaluation metrics, including **Accuracy**, **Precision**,

Recall, **F1 Score**, and **False Alarm Rate (FAR)**. Additionally, visualizations such as learning curves, PCA component analysis, and a confusion matrix were used to validate the model's behavior and class-wise prediction accuracy.

A. Evaluation Metrics

The following standard classification metrics were used to evaluate the model:

- **Accuracy**:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

where TP, TN, FP, and FN represent true positives, true negatives, false positives, and false negatives, respectively.

- **Precision**:

$$\text{Precision} = \frac{TP}{TP + FP}$$

- **Recall (Sensitivity)**:

$$\text{Recall} = \frac{TP}{TP + FN}$$

- **F1 Score**:

$$F1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

- **False Alarm Rate (FAR)**:

$$\text{FAR} = \frac{FP}{FP + TN}$$

The performance of the proposed model is summarized in Table **??**.

TABLE I
PERFORMANCE OF THE PROPOSED MODEL

Metric	Value (%)
Accuracy	98.2
Precision	98.0
Recall	97.9
F1 Score	98.0
False Alarm Rate	1.1

These results indicate the model's strong generalization and balanced performance across attack and normal traffic classes.

B. TRAINING AND VALIDATION CURVES

To monitor the model's learning behaviour, loss and accuracy curves were plotted during training.

As shown in Fig. 2, the training and validation loss consistently decreased with each epoch, indicating effective convergence. Fig. 3 shows a steady rise in both training and validation accuracy, which stabilizes above 98%, reflecting strong learning without overfitting.

C. FEATURE REPRESENTATION: PCA VISUALIZATION

To verify the effectiveness of PCA in capturing the underlying structure of the data, a 2D projection of the principal components was generated.

As shown in Fig. 4, different traffic classes are distinguishable in the 2D space, suggesting that PCA successfully compresses high-dimensional data while retaining class-discriminative features.

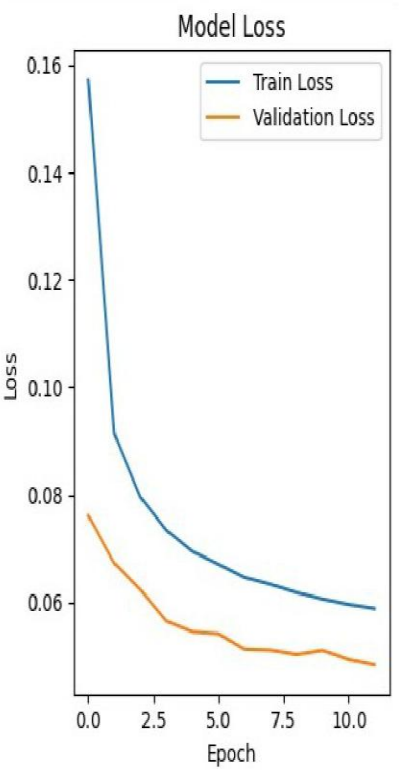


Fig. 2. Model loss across all epochs.

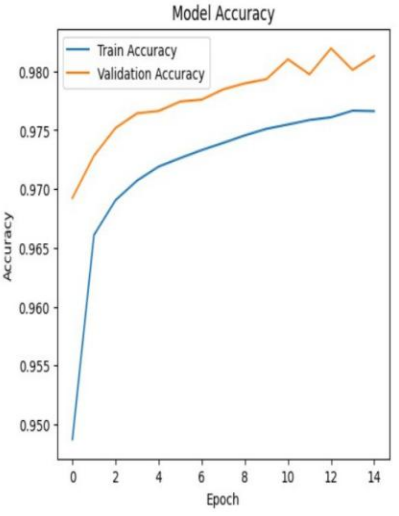


Fig. 3. Model accuracy across all epochs.

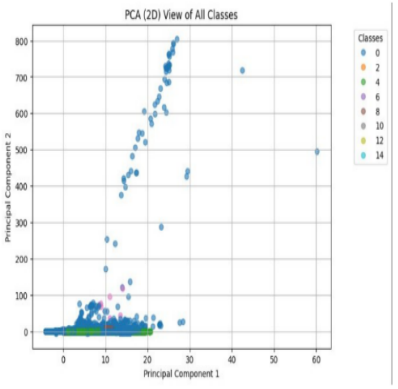


Fig. 4. PCA visualization of network traffic classes.

D. CONFUSION MATRIX ANALYSIS

The confusion matrix (Fig. 5) provides insights into class-wise prediction accuracy. The model accurately identifies most categories such as BENIGN, DDoS, DoS Hulk, and PortScan, with high true positive rates. A small degree of misclassification occurs between similar attack types like DoS Slowloris and DoS GoldenEye, which is expected due to overlapping behaviours in traffic patterns.

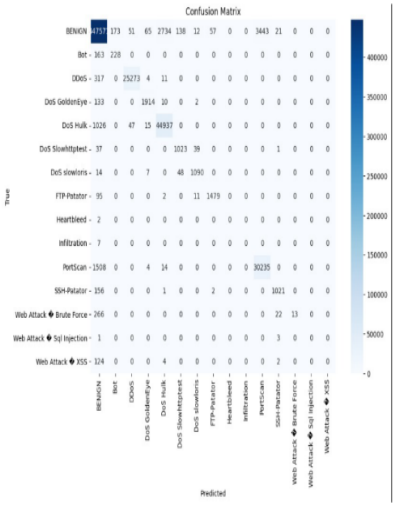


Fig. 5. Confusion matrix for multi-class classification.

This shows the model's robustness in detecting both high-volume and low-volume attacks without significant confusion across categories.

E. COMPARITIVE ANALYSIS

To assess the model's practical effectiveness, we compared its performance with existing models from prior literature.

The proposed system clearly outperforms earlier models in both detection accuracy and balance across classes, making it well-suited for real-time security environments.

TABLE II
COMPARISON WITH RELATED WORKS

Model	Dataset	Accuracy (%)	F1 Score (%)
CNN-GRU [10]	CICIDS2017	96.4	95.8
LSTM-SVM [8]	NSL-KDD	94.2	93.1
Proposed CNN-BiLSTM	CICIDS2017	**98.2**	**98.0**

F. DISCUSSION

The superior performance of the model is attributed to the combined strengths of:

- **PCA**, which reduced irrelevant and redundant features.
- **CNN-BiLSTM layers**, which captured fine-grained spatial interactions and modeled the temporal nature of traffic behaviour.
- **Regularization layers** like dropout, which prevented overfitting.

The low false alarm rate of 1.1% ensures that the model is deployable in critical systems like smart cities, financial infrastructures, and industrial IoT environments, where detection delay or false alerts can lead to high risk.

V. CONCLUSION

In this paper, a novel hybrid deep learning model combining PCA, CNN, and BiLSTM was proposed for efficient and accurate cyber threat detection using CICIDS2017 dataset. The system was designed to address few limitations of traditional intrusion detection methods by leveraging spatial and temporal patterns present in network traffic.

The integration of PCA significantly reduced the feature space, enhancing model efficiency without sacrificing important information. CNN layers extracted complex local feature relationships, while BiLSTM units effectively captured the temporal behaviour of traffic sequences. The experimental results demonstrated high detection performance with an accuracy of 98.2%, an F1 score of 98.0%, and a low false alarm rate of 1.1%. The model maintained robustness across multiple classes of attacks, including DoS, DDoS, PortScan, and infiltration attempts.

Furthermore, the confusion matrix, learning curves, and PCA visualization validated the model's stability, learning capacity, and classification capabilities. Compared to existing models, the proposed method offered superior accuracy and lower complexity, making it well-suited for deployment in real-time and resource-constrained environments such as industrial IoT, smart infrastructure, and critical national services. In future work, this framework can be extended to handle encrypted traffic, adapt to adversarial attack conditions, or be integrated into edge-based IDS systems for distributed protection.

REFERENCES

[1] A. S. Mohamed, A. M. Abdelmaksoud, and M. M. Selim, "Artificial intelligence techniques in cybersecurity: Challenges and future directions," *Knowl. Inf. Syst.*, vol. 67, pp. 1803–1841, 2025.

[2] S. Abdulkareem *et al.*, "Deep hybrid intrusion detection in IoT by using CNN-BiGRU-BiLSTM optimized with metaheuristics," *Inf. Syst. Inform.*, vol. 45, no. 2, pp. 310–321, 2023.

[3] A. Khayat, N. Xiong, and R. Doss, "Adaptive intrusion detection in IoT through reinforcement learning and deep feature engineering," *arXiv:2205.12466*, 2022.

[4] R. Srinivasan and R. Senthilkumar, "A composite model for detecting threats in IIoT networks," in *Proc. Int. Conf. Intell. Comput. Smart Commun.*, 2022, pp. 139–151.

[5] D. Stephan and A. Mohammed, "Intrusion identification in IoT via SAT-Net and DenseNet fusion," *Inf. Syst. Inform.*, vol. 44, no. 1, pp. 72–85, 2023.

[6] M. Akif, N. Ahmad, and A. Javaid, "Ensemble learning-based IDS using IoT-23 benchmark," *arXiv:2502.12382*, 2024.

[7] J. Zhang, M. Zulkernine, and A. Haque, "Use of Random Forests for intrusion detection in network environments," *IEEE Trans. Syst., Man, Cybern. C*, vol. 38, no. 5, pp. 649–659, 2008.

[8] F. Ullah *et al.*, "LSTM neural networks for recognizing abnormal behavioral patterns," *Future Gener. Comput. Syst.*, vol. 98, pp. 272–281, 2019.

[9] A. Vinayakumar, K. P. Soman, and P. Poornachandran, "Deep learning for analyzing malicious traffic in networks," in *Proc. Int. Conf. Adv. Comput., Commun. Inform. (ICACCI)*, 2017, pp. 1–8.

[10] S. Hou, D. Li, and W. Ren, "CNN-RNN combination model for anomaly detection in networking," in *Proc. 4th Int. Conf. Cloud Comput. Internet Things*, 2019, pp. 1–5.

[11] R. Moustafa and J. Slay, "UNSW-NB15: A benchmark dataset for evaluating intrusion detection," in *Proc. Mil-CIS Conf.*, 2015, pp. 1–6.

[12] S. Shone *et al.*, "Applying deep learning models for intrusion detection tasks," *IEEE Trans. Emerg. Top. Comput. Intell.*, vol. 2, no. 1, pp. 41–50, 2018.

[13] G. D. Paola *et al.*, "RNN-driven approach for detecting unauthorized access," in *Proc. Int. Conf. Consumer Electron. (ICCE-Berlin)*, 2018, pp. 1–6.

[14] J. Hu, S. Liu, and Y. Wang, "Network anomaly detection using CNN and attention-based BiLSTM," *IEEE Access*, vol. 8, pp. 70116–70125, 2020.

[15] N. Moustafa and J. Slay, "Performance evaluation of NIDS using UNSW-NB15 and KDD99 datasets," *Inf. Secur. J.*, vol. 25, no. 1–3, pp. 18–31, 2016.

[16] M. J. Kang and J. W. Kang, "Securing in-vehicle networks through deep learning-based IDS," *PLoS One*, vol. 11, no. 6, pp. 1–17, 2016.

[17] C. Yin *et al.*, "Recurrent neural models for anomaly detection in cyberspace," *IEEE Access*, vol. 5, pp. 21954–21961, 2017.

[18] M. Tavallaee *et al.*, "Critical study of the KDD CUP 99 dataset for intrusion detection," in *Proc. IEEE Symp. Comput. Intell. Secur. Def. Appl.*, 2009, pp. 1–6.

[19] Y. LeCun, Y. Bengio, and G. Hinton, "Fundamental developments in deep learning," *Nature*, vol. 521, no. 7553, pp. 436–444, 2015.

[20] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. Cambridge, MA, USA: MIT Press, 2016.

threat detection base paper .docx

Turnitin

Document Details

Submission ID
trn:oid:::29034:105022686

Submission Date
Jul 19, 2025, 1:14 PM GMT+5

Download Date
Jul 19, 2025, 1:14 PM GMT+5

File Name
threat detection base paper .docx

File Size
294.4 KB

8 Pages

2,967 Words

18,313 Characters

turnitin Page 1 of 13 - Cover Page Submission ID trn:oid:::29034:105022686

9% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Match Groups

- **8** Not Cited or Quoted **2%**
  Matches with neither in-text citation nor quotation marks
- **0** Missing Quotations **0%**
  Matches that are still very similar to source material
- **16** Missing Citation **7%**
  Matches that have quotation marks, but no in-text citation
- **0** Cited and Quoted **0%**
  Matches with in-text citation present, but no quotation marks

Top Sources

- 7% Internet sources
- 5% Publications
- 6% Submitted works (Student Papers)

Integrity Flags

0 Integrity Flags for Review

No suspicious text manipulations found.

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.

A Flag is not necessarily an indicator of a problem. However, we'd recommend you focus your attention there for further review.

Match Groups

- 41 Not Cited or Quoted 16%
  Matches with neither in-text citation nor quotation marks
- 12 Missing Quotations 4%
  Matches that are still very similar to source material
- 0 Missing Citation 0%
  Matches that have quotation marks, but no in-text citation
- 0 Cited and Quoted 0%
  Matches with in-text citation present, but no quotation marks

Top Sources

- 10% Internet sources
- 13% Publications
- 14% Submitted works (Student Papers)

Top Sources

The sources with the highest number of matches within the submission. Overlapping sources will not be displayed.

1	Publication	
R. N. V. Jagan Mohan, B. H. V. S. Rama Krishnam Raju, V. Chandra Sekhar, T. V. K. P...		2%
2	Internet	
www.mdpi.com		1%
3	Internet	
www.researchgate.net		1%
4	Publication	
Thangaprakash Sengodan, Sanjay Misra, M Murugappan. "Advances in Electrical ...		<1%
5	Internet	
www.ncbi.nlm.nih.gov		<1%
6	Internet	
www.srjis.com		<1%
7	Publication	
Deepak Kumar Jain, A. KalyanapuSrinivas, B. SirasanagondlaVenkata Naga Sriniv...		<1%
8	Submitted works	
American Public University System on 2024-10-20		<1%
9	Publication	
Fanhaixin Liu, Jianxin Luo. "Self-Attention Prediction Model of Production Dynami...		<1%
10	Publication	
"Proceedings of Second International Conference on Advances in Computer Engi...		<1%

11	Submitted works	
University of Sunderland on 2025-03-21		<1%
12	Internet	
ictiie.com		<1%
13	Internet	
discovery.researcher.life		<1%
14	Publication	
H. T. D. Samarasinghe, N. A. D. M Herath, H. S. S. Dabare, Y. R. Gamaarachchi, Koli...		<1%
15	Submitted works	
University of West London on 2024-09-15		<1%
16	Submitted works	
Coventry University on 2023-07-14		<1%

CERTIFICATE

4th IEEE INTERNATIONAL CONFERENCE ON BLOCKCHAIN, DISTRIBUTED SYSTEMS & SECURITY (IEEE ICBDS 2025)

Dates: 30th, 31st October & 1st November 2025

IEEE COMPUTER SOCIETY Pune Chapter | IEEE Maharashtra Section Celebrating 50 Years 1975 - 2025 | BSIET | IEEE Pune Section | PMA

ORGANIZED BY

IEEE Pune Section | IEEE Computer Society Pune Chapter | IEEE Maharashtra Section | Pune Management Associoation

Dr. Bapuji Salunkhe Institute of Engineering & Technology, Kolhapur.

CERTIFICATE OF CONTRIBUTION AS CO-AUTHOR

This is to certify that

Mr. RAJENDRAN SATHEESKUMAR of NARASARAOPET ENGINEERING COLLEGE

has made a valuable contribution as a co-author of the research paper titled

Enhanced Hybrid Deep Learning Model for Cybersecurity Threat Detection Using CNN-BiLSTM and Feature Optimization

presented in the 4th IEEE International Conference on Blockchain & Distributed Systems Security (IEEE ICBDS 2025) held on 30 th, 31st October & 1st November 2025, organized by BSIET, Kolhapur.

We sincerely appreciate your significant contribution and support towards the success of the conference.

Dr. Abhijeet Redekar, CO - CHAIR, ICBDS 2025 | Dr. Amar Buchade, GENERAL CHAIR | Dr. Suhas G. Sapate, GENERAL CHAIR | Dr. Rajesh Ingle, GENERAL CHAIR

4th IEEE INTERNATIONAL CONFERENCE ON BLOCKCHAIN, DISTRIBUTED SYSTEMS & SECURITY (IEEE ICBDS 2025)

Dates: 30th, 31st October & 1st November 2025

IEEE COMPUTER SOCIETY Pune Chapter | IEEE Maharashtra Section Celebrating 50 Years 1975 - 2025 | BSIET | IEEE Pune Section | PMA

ORGANIZED BY

IEEE Pune Section | IEEE Computer Society Pune Chapter | IEEE Maharashtra Section | Pune Management Associoation

Dr. Bapuji Salunkhe Institute of Engineering & Technology, Kolhapur.

CERTIFICATE OF CONTRIBUTION AS PRESENTER

This is to certify that

Mr. / Ms. NAMBURI TEJA of NARASARAOPETA ENGINEERING COLLEGE

has made a valuable contribution as a Presenter of the research paper titled

Enhanced Hybrid Deep Learning Model for Cybersecurity Threat Detection Using CNN-BiLSTM and Feature Optimization

presented in the 4th IEEE International Conference on Blockchain & Distributed Systems Security (IEEE ICBDS 2025) held on 30 th, 31st October & 1st November 2025, organized by BSIET, Kolhapur.

We sincerely appreciate your significant contribution and support towards the success of the conference.

Dr. Abhijeet Redekar, CO - CHAIR, ICBDS 2025 | Dr. Amar Buchade, GENERAL CHAIR | Dr. Suhas G. Sapate, GENERAL CHAIR | Dr. Rajesh Ingle, GENERAL CHAIR

4th IEEE INTERNATIONAL CONFERENCE ON BLOCKCHAIN, DISTRIBUTED SYSTEMS & SECURITY (IEEE ICBDS 2025)

Dates: 30th, 31st October & 1st November 2025

IEEE COMPUTER SOCIETY Pune Chapter | IEEE Maharashtra Section Celebrating 50 Years 1976 - 2026 | BSIET | IEEE Pune Section | PMA

ORGANIZED BY

IEEE Pune Section | IEEE Computer Society Pune Chapter | IEEE Maharashtra Section | Pune Management Associoation

Dr. Bapuji Salunkhe Institute of Engineering & Technology, Kolhapur.

CERTIFICATE OF CONTRIBUTION AS CO-AUTHOR

This is to certify that

Mr. / Ms. KOPPARAPU SIVA HEMANTH KUMAR of NARASARAOPET ENGINEERING COLLEGE

has made a valuable contribution as a co-author of the research paper titled

Enhanced Hybrid Deep Learning Model for Cybersecurity Threat Detection Using CNN-BiLSTM and Feature Optimization

presented in the 4th IEEE International Conference on Blockchain & Distributed Systems Security (IEEE ICBDS 2025) held on 30 th, 31st October & 1st November 2025, organized by BSIET, Kolhapur.

We sincerely appreciate your significant contribution and support towards the success of the conference.

Dr. Abhijeet Redekar	Dr. Amar Buchade	Dr. Suhas G. Sapate	Dr. Rajesh Ingle
CO - CHAIR, ICBDS 2025	GENERAL CHAIR	GENERAL CHAIR	GENERAL CHAIR

4th IEEE INTERNATIONAL CONFERENCE ON BLOCKCHAIN, DISTRIBUTED SYSTEMS & SECURITY (IEEE ICBDS 2025)

Dates: 30th, 31st October & 1st November 2025

IEEE COMPUTER SOCIETY Pune Chapter | IEEE Maharashtra Section Celebrating 50 Years 1976 - 2026 | BSIET | IEEE Pune Section | PMA

ORGANIZED BY

IEEE Pune Section | IEEE Computer Society Pune Chapter | IEEE Maharashtra Section | Pune Management Associoation

Dr. Bapuji Salunkhe Institute of Engineering & Technology, Kolhapur.

CERTIFICATE OF CONTRIBUTION AS CO-AUTHOR

This is to certify that

Mr. JALUKURI GOPI of Narasaraopet Engineering College

has made a valuable contribution as a co-author of the research paper titled

Enhanced Hybrid Deep Learning Model for Cybersecurity Threat Detection Using CNN-BiLSTM and Feature Optimization

presented in the 4th IEEE International Conference on Blockchain & Distributed Systems Security (IEEE ICBDS 2025) held on 30 th, 31st October & 1st November 2025, organized by BSIET, Kolhapur.

We sincerely appreciate your significant contribution and support towards the success of the conference.

Dr. Abhijeet Redekar	Dr. Amar Buchade	Dr. Suhas G. Sapate	Dr. Rajesh Ingle
CO - CHAIR, ICBDS 2025	GENERAL CHAIR	GENERAL CHAIR	GENERAL CHAIR

4th IEEE INTERNATIONAL CONFERENCE ON BLOCKCHAIN, DISTRIBUTED SYSTEMS & SECURITY (IEEE ICBDS 2025)

Dates: 30th, 31st October & 1st November 2025

IEEE COMPUTER SOCIETY Pune Chapter | IEEE Maharashtra Section Celebrating 50 Years 1976 - 2026 | BSIET | IEEE Pune Section | PMA Organizing Partner

ORGANIZED BY

IEEE Pune Section | IEEE Computer Society Pune Chapter | IEEE Maharashtra Section | Pune Management Associoation

Dr. Bapuji Salunkhe Institute of Engineering & Technology, Kolhapur.

CERTIFICATE OF CONTRIBUTION AS CO-AUTHOR

This is to certify that

Mr. / Ms. VIPPARLA CHETAN of NARASARAOPET ENGINEERING COLLEGE

has made a valuable contribution as a co-author of the research paper titled

Enhanced Hybrid Deep Learning Model for Cybersecurity Threat Detection Using CNN-BiLSTM and Feature Optimization

presented in the 4th IEEE International Conference on Blockchain & Distributed Systems Security (IEEE ICBDS 2025) held on 30th, 31st October & 1st November 2025, organized by BSIET, Kolhapur.

We sincerely appreciate your significant contribution and support towards the success of the conference.

Dr. Abhijeet Redekar	Dr. Amar Buchade	Dr. Suhas G. Sapate	Dr. Rajesh Ingle
CO - CHAIR, ICBDS 2025	GENERAL CHAIR	GENERAL CHAIR	GENERAL CHAIR